

R E P O R T

3. 쓰레드, 소켓 및 GUI 프로그래밍



과목명	오픈소스 프로젝트
담당교수	김성우 교수님
학과	컴퓨터소프트웨어공학과
학번	20212979
이름	임진호



동의대학교
DONG-EUI UNIVERSITY

목 차

I . 실습에 필요한 준비사항 (이론)	3~23
1. 교재에서 쓰레드에 대한 내용을 읽고 이해한다	3~6
2. 리눅스에서 쓰레드 동기화 및 통신 기법	6~13
3. 소켓 프로그래밍에 대한 내용 정리	13~19
4. 리눅스에서 사용하는 GUI 툴킷에 대한 내용을 조사하여 보고서에 정리	19~23
 II . 실습 결과	 24~51
1. github 저장소에 업로드	24
2. 쓰레드 관련 함수들을 사용하여 익숙해진다.	24~28
3. 쓰레드를 사용하여 생산자 소비자 문제를 해결하는 제한 버퍼 생성 및 활용 구현	28~30
4. 뮤텝스와 조건변수를 사용한 부모, 자식 간 통신	30~32
5. 소켓을 이용하여 프로그램을 작성 및 실행	32~37
6. select를 이용한 다중 클라이언트 처리 채팅 프로그램	37~40
7. TCP 소켓을 이용한 HTTP GET 및 POST 메소드, CGI 프로그램 실행을 구현하는 웹서버	40~45
8. GUI 관련 함수들을 사용하여 프로그램을 작성 및 실행	45~47
9. GTK+ 또는 Qt 를 이용하여 간단한 계산기 프로그램을 작성하	48~51
 II . 검토	 52

1. 교재에서 쓰레드에 대한 내용을 읽고 이해한다.

1.1 쓰레드란?

프로그램의 실행 흐름을 나타내는 기본 단위를 의미한다.

프로세스는 보통 하나의 쓰레드를 가지지만, 새로운 작업 모델을 적용하여 여러 개의 쓰레드를 가질 수 있다. 이처럼 프로세스의 프로그램, 데이터 영역 등의 주소 공간은 서로 공유하고, 스택 등만 따로 가지는 새로운 작업 개체를 쓰레드(thread) 라고 일컫는다

- 프로세스 - 중량 프로세스

- 쓰레드 - 경량 프로세스

1. 주소 공간(프로그램, 데이터 영역) 공유

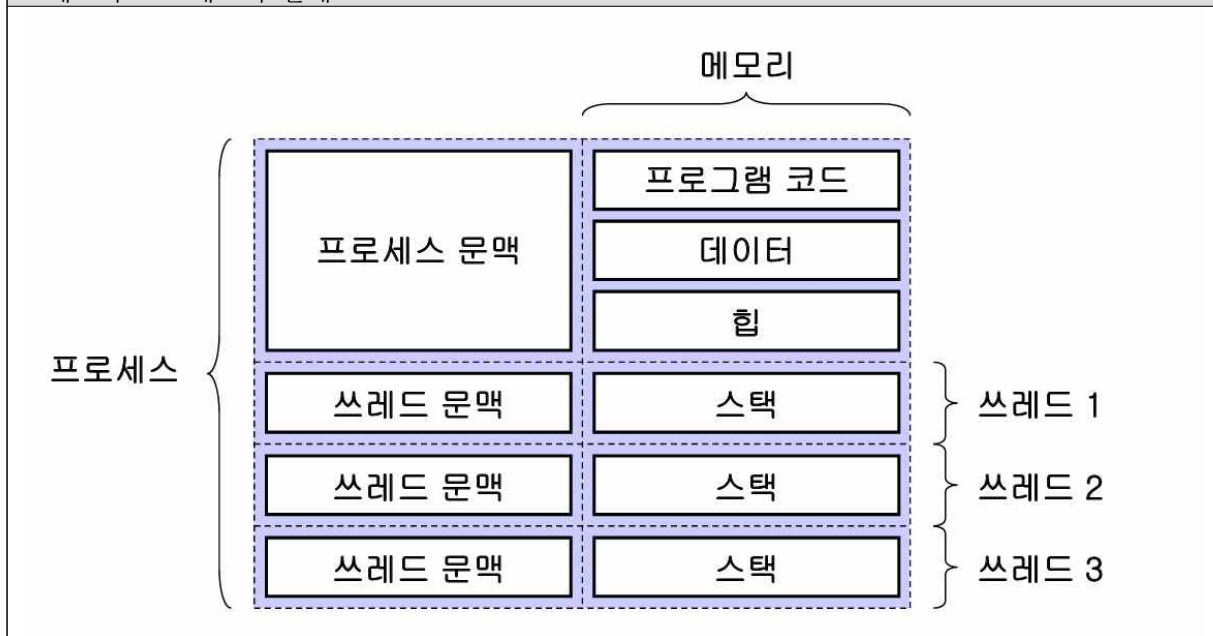
- 프로세스 지시사항, 대부분의 데이터, open 파일들, 시그널과 핸들러, 사용자와 그룹ID 등

2. 스택, 쓰레드 문맥(프로그램 카운터등)만 따로 가짐

- 쓰레드 ID, 프로그램 카운터등의 문맥 정보, 스택, errno, 시그널 마스크, 우선순위

3. 프로세스에 비해 쓰레드 간 통신이 간편하고, 생성 시간도 짧은 장점을 가짐

쓰레드와 프로세스의 관계



1.2 POSIX 쓰레드 개요

POSIX (Portable Operating System Interface)

: UNIX 기반 운영체제를 위한 포터블 인터페이스에 관한 국제 표준

- POSIX Thread

1. POSIX 표준 중 쓰레드에 관한 API 부분을 말함

- IEEE POSIX 1003.1c (1995)

2. 구성요소

- 쓰레드 관리
- 뮤텝스, 조건변수

3. 줄여서 pthread 라고도 함 - 모든 쓰레드 함수 이름이 pthread_로 시작

접두사	의미
pthread_	쓰레드와 기타 함수
pthread_attr_	쓰레드 속성 개체
pthread_mutex_	뮤텝스
pthread_mutexattr_	뮤텝스 속성 개체
pthread_cond_	조건변수
pthread_condattr_	조건변수 속성 개체
pthread_key	쓰레드 특정-자료 키

1.3 쓰레드 ID와 참조

POSIX 쓰레드는 pthread_t 자료형의 쓰레드ID를 통해 참조됨

• 쓰레드 참조 함수들

기능

-pthread_self() : 자신의 쓰레드ID 획득

-pthread_equal() : 두 개의 쓰레드ID를 비교

사용법

```
#include <pthread.h>
```

```
int pthread_self(void);
```

```
int pthread_equal(pthread_t t1, pthread_t t2);
```

- pthread_self 함수는 성공적인 호출에 대하여, 자신의 쓰레드ID 반환
- pthread_equal 함수는 두 인자가 같으면 0 이 아닌 값, 다르면 0을 반환

1.4 쓰레드 생성과 종료

POSIX (Portable Operating System Interface)

: UNIX 기반 운영체제를 위한 포터블 인터페이스에 관한 국제 표준

pthread_create 사용법 (쓰레드 생성)

```
#include <pthread.h>
```

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
```

```
void *(*start)(void *), void *arg);
```

- 성공적인 호출에 대하여 0을 반환하고, 실패하면 오류코드 반환

pthread_exit 사용법 (호출하는 쓰레드를 종료)

```
#include <pthread.h>
```

```
void pthread_exit(void *retval);
```

반환값 없음

1.5 쓰레드 분리와 결합

pthread_detach()함수와, pthread_join()함수

pthread_detach() 사용법 (쓰레드 분리)

```
#include <pthread.h>
```

```
int pthread_detach (pthread_t thread);
```

- 쓰레드를 분리하여 분리 상태로 둔다.
- 성공적인 호출: 0, 그렇지 않을 경우: 0이 아닌값을 반환한다.

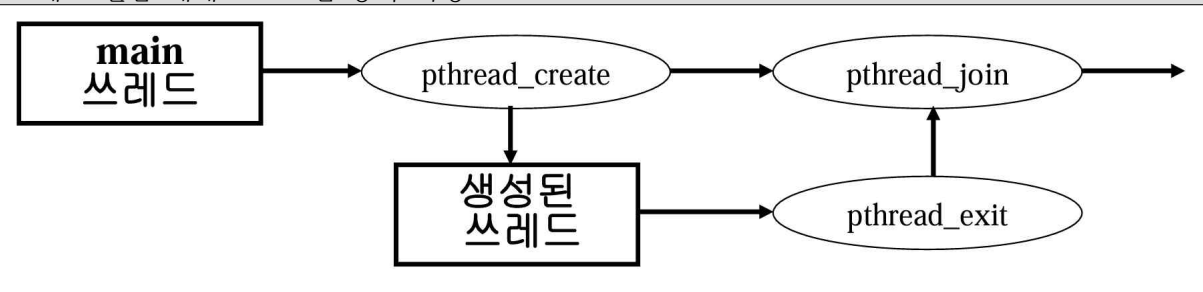
pthread_join() 사용법 (쓰레드 결합)

```
#include <pthread.h>
```

```
int pthread_join (pthread_t thread, void **thread_return);
```

- 쓰레드의 종료를 기다린다
- 성공적인 호출인 경우 0, 수행이 실패한 경우 0이 아닌 값을 반환한다.

쓰레드 결합 예제 프로그램 동작 과정



1.6 쓰레드 취소

- 한 쓰레드는 다른 쓰레드를 종료하도록 취소 요청 가능
- 취소 요청을 받은 쓰레드는 설정 상태에 따라 동작

취소 유형	취소 상태	동작
모두	비활성화 (disabled)	취소 활성화될 때까지 보류된다.
지연 (deferred)	활성화 (enabled)	다음 취소 지점에 도달할 때까지 보류된다.
비동기 (asynchronous)	활성화 (enabled)	모든 취소 요청이 즉시 처리된다.

• 취소지점

- 쓰레드 취소를 허용하는 프로그램의 특정 부분

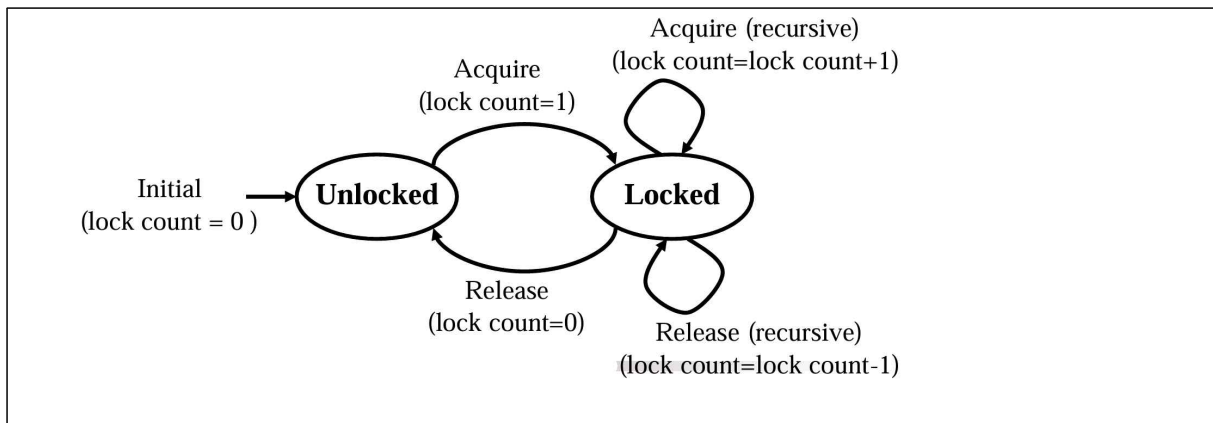
-리눅스에서pthread_join(), pthread_cond_wait(), pthread_cond_timedwait(), pthread_testcancel(), sem_wait(), sigwait()와 같은POSIX 쓰레드 함수들과 read()와 같은 일부 시스템함수들은 취소 지점 포함

1.6 쓰레드 취소	
형식	<pre>#include <pthread.h> int pthread_cancel(pthread_t thread); int pthread_setcancelstate(int state, int *oldstate); int pthread_setcanceltype(int type, int *oldtype); void pthread_testcancel(void)</pre>
기능	• pthread_cancel - 쓰레드를 취소 • pthread_setcancelstate - 쓰레드 취소 상태 변경 • pthread_setcanceltype - 쓰레드 취소 유형 변경 • pthread_testcancel - 쓰레드 취소 요청 점검
변환값	성공적 수행 : 0, 그렇지않을 경우 : 0이 아닌값

2. 리눅스에서 쓰레드 동기화 및 통신 기법

2.1 쓰레드 동기화	
• 쓰레드 간 공유 상태 변경 1. 공유하는 자원을 변경하기 위해 안전하게 보호하는 도구 필요 → 뮤텝스, 조건변수, 읽기쓰기락, 원자형 정수 2. 쓰레드 간 자원을 공유하는 기능 필요 3. 뮤텝스 사용 예	

2.2 뮤텝스	
• Mutual Exclusion (Mutex) 의 약자 • 공유하는 자원을 보호하기 위한 짧은 잠금(lock) 장치 • 소유권, 재귀 잠금, 쓰레드 삭제 안전성, 우선순위 역전 문제 회피 기능 포함 • 2가지 상태를 가짐 : Unlocked(0) or locked(1)	
세마포어와의 차이점	
• 공유 자원에 대해 상호배제적으로 접근 → 계수 세마포어는 여러 개의 자원을 공유 • 소유권을 가져 뮤텝스를 획득한 쓰레드가 뮤텝스를 해제 → 이진 세마포어는 임의의 프로세스/쓰레드가 세마포어 해제 가능	



속성

- 뮤텝스 소유권
 - 쓰레드가 뮤텝스를 처음 잠글 때 뮤텝스를 획득
 - 뮤텝스를 획득할 때, 다른 태스크가 뮤텝스를 잠그거나 풀 수 없다.
- 재귀 잠금
 - 잠금 상태에서 여러 번 뮤텝스를 획득하도록 허용
 - 계수는 뮤텝스를 소유한 쓰레드가 뮤텝스를 잠그거나 풀 횟수를 추적하는데 사용
- 쓰레드 삭제 안전성
 - 쓰레드가 뮤텝스를 잠그거나 풀 때 쓰레드 삭제가 잠겨진다.
- 우선순위 역전 회피
 - 더 높은 우선순위 쓰레드가 더 낮은 우선 순위 쓰레드가 사용하는 자원을 위해대기 할 때 우선순위 역전 발생.

뮤텝스 생성 및 파괴 함수

사용법	<pre> #include <pthread.h> pthread_mutex_t mutex=PTHREAD_MUTEX_INITIALIZER; int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *mutexattr); int pthread_mutex_destroy(pthread_mutex_t *mutex); </pre>
기능	- 뮤텝스를 초기화한다 - 뮤텝스를 동적으로 생성하고 초기화 한다 - 뮤텝스를 파괴한다
변환값	- pthread_mutex_init 은 항상 0 을 반환 - pthread_mutex_destroy 는 성공적 수행: 0, 그렇지 않을 경우: 0이아닌값

mutex 잠금 및 해제 함수	
사용법	<pre>#include <pthread.h> int pthread_mutex_lock(pthread_mutex_t *mutex); int pthread_mutex_trylock(pthread_mutex_t *mutex); int pthread_mutex_unlock(pthread_mutex_t *mutex);</pre>
기능	- pthread_mutex_lock - 뮤텝스를 잠금 - pthread_mutex_trylock - 뮤텝스를 잠그지만, 즉시 반환 - pthread_mutex_unlock - 뮤텝스를 해제
변환값	성공적 수행: 0, 그렇지 않을 경우: 0이 아닌값
주의점	- Pthread_mutex_lock 는 뮤텝스가 다른 쓰레드에 의해 잠겨져 있는 상태 이면 해제 될 때까지 호출 쓰레드를 대기 시키지만, pthread_mutex_trylock 는 즉시 반환

2.4 원자적 연산 (Atomic Operations)
락(Lock)을 사용하지 않고 하드웨어(CPU)의 원자적 명령어를 사용하여 데이터 경쟁을 해결하는 동기화 메커니즘 카운터 증가, 플래그 설정 등 단순하지만 빈번한 변수 조작에 고성능을 내기 위해 사용
• 뮤텝스와의 차이점 (Lock-free) - Non-blocking: 락을 획득하기 위해 대기(Sleep)하지 않고 즉시 실행되거나 실패함 (문맥 교환 비용 없음) - Deadlock Free: 락을 걸지 않으므로 교착 상태(Deadlock)가 원천적으로 발생하지 않음 • CAS (Compare-And-Swap) 알고리즘 - 비교 후 교환: "메모리의 현재 값이 내가 예상한 값(Expected)과 같다면, 새로운 값(Desired)으로 변경하라"는 논리를 하나의 명령어로 처리 - 낙관적 동기화: 충돌이 드물 것이라 가정하고 수행하며, 실패 시(값이 다를 시) 루프를 돌며 재시도함

원자적 연산 사용법	
형식	<pre>#include <stdatomic.h> void atomic_init(volatile atomic_int *obj, int value); void atomic_store(volatile atomic_int *obj, int desired); int atomic_load(volatile atomic_int *obj); int atomic_fetch_add(volatile atomic_int *obj, int arg); _Bool atomic_compare_exchange_strong(volatile atomic_int *obj, int *expected, int desired);</pre>

기능	- atomic_init – 원자적 변수를 초기화 (락 없이 안전하게 초기값 설정) - atomic_store – 원자적 변수에 값을 저장 (쓰기) - atomic_load – 원자적 변수의 값을 읽어옴 (읽기) - atomic_fetch_add – 변수의 값을 원자적으로 증가시킴 (i++와 유사하나 스레드 안전함) - atomic_compare_exchange_strong – CAS 알고리즘 수행 (예상값과 일치하면 새 값으로 교체)
변환값	- atomic_load: 현재 저장된 값 - atomic_fetch_add: 연산 수행 이전의 원본 값 - atomic_compare_exchange_strong: 성공 시 true, 실패 시 false

2.5 스핀락 (Spinlock)

임계 영역(Critical Section)에 진입할 때까지 스레드가 문맥 교환(Context Switch) 없이 루프를 돌며 대기(Busy-waiting)하는 동기화 메커니즘 멀티코어 프로세서에서 임계 영역이 매우 짧아, 잠드는 비용보다 기다리는 비용이 더 적게 들 때 주로 사용

- 뮤텍스와 차이점
 - Busy-Waiting: 뮤텍스는 잠금을 못 얻으면 대기 상태(Sleep)로 전환되지만, 스핀락은 CPU를 계속 점유하며 락이 풀릴 때까지 검사를 반복함
 - Overhead: 문맥 교환 오버헤드가 없어서 짧은 작업에는 빠르지만, 오랫동안 락을 잡고 있으면 CPU 자원을 낭비함

스핀락 함수 사용법

형식	<pre>#include <pthread.h> int pthread_spin_init(pthread_spinlock_t *lock, int pshared); int pthread_spin_lock(pthread_spinlock_t *lock); int pthread_spin_trylock(pthread_spinlock_t *lock); int pthread_spin_unlock(pthread_spinlock_t *lock); int pthread_spin_destroy(pthread_spinlock_t *lock);</pre>
기능	- pthread_spin_init – 스핀락 객체를 초기화 (pshared로 프로세스 간 공유 여부 설정) - pthread_spin_lock – 스핀락 잠금 획득 시도 (획득할 때까지 무한 루프) - pthread_spin_trylock – 스핀락 잠금 시도 (실패 시 기다리지 않고 즉시 에러 반환) - pthread_spin_unlock – 스핀락 잠금 해제 - pthread_spin_destroy – 스핀락 객체 및 리소스 제거
변환값	성공적 수행: 0, 그렇지 않을 경우: 0이 아닌 에러 코드

2.6 Eventfd	
리눅스 커널 2.6.22부터 도입된 리눅스 고유의 통신 기법 파일 디스크립터(File Descriptor)를 생성하여, 이를 통해 64비트 정수형 카운터 값을 주고받으며 쓰레드(또는 프로세스) 간에 이벤트를 전달함	
- 통신으로서의 특징 - 가벼운 알림: 파이프(Pipe)보다 메모리 오버헤드가 훨씬 적어 고성능 이벤트 통신에 유리 - 범용성: 파일 디스크립터를 사용하므로 select, poll, epoll 같은 I/O 멀티플렉싱과 연동하여 여러 이벤트를 한 번에 관리하기 쉬움	

Eventfd 함수 사용법	
형식	<pre>#include <sys/eventfd.h> #include <unistd.h> int eventfd(unsigned int initval, int flags); ssize_t read(int fd, void *buf, size_t count); ssize_t write(int fd, const void *buf, size_t count); int close(int fd);</pre>
기능	- eventfd : 통신을 위한 이벤트 파일 디스크립터 생성 (초기값 설정 가능) - write : 8바이트 정수 값을 파일에 씀 (카운터 증가 = 이벤트 발생 신호) - read : 파일에서 8바이트 정수 값을 읽음 (카운터 읽기 및 초기화 = 이벤트 수신) - close : 파일 디스크립터 닫기
변환값	eventfd: 성공 시 파일 디스크립터(양수), 실패 시 -1 read/write: 성공 시 읽거나 쓴 바이트 수(8), 실패 시 -1

2.7 읽기-쓰기 잠금	
- 잠그는 동안 여러 쓰레드가 동시에 읽을수는 있지만, 하나의 쓰레드만 쓰기가 가능한 동기화 매커니즘 - 데이터베이스와 같은 공유 자료 개체에 접근하기 위해 사용	
뮤텍스와의 차이점	
- 공유 읽기 접근(shared read access) - 읽기 잠금을 수행하는 동안 읽기 쓰레드는 접근을 허용하고 쓰기 쓰레드는 허용 하지 않음 - 배타적 쓰기 접근(exclusive write access) - 쓰기 잠금을 수행하는 동안 다른 쓰레드의 접근을 허용 하지 않음	

읽기-쓰기 잠금 및 해제 함수	
형식	<pre>#include <pthread.h> int pthread_rwlock_rdlock(pthread_rwlock_t *rwlock); int pthread_rwlock_tryrdlock(pthread_rwlock_t *rwlock); int pthread_rwlock_wrlock(pthread_rwlock_t *rwlock); int pthread_rwlock_trywrlock(pthread_rwlock_t *rwlock); int pthread_rwlock_unlock(pthread_rwlock_t *rwlock);</pre>
기능	- pthread_rwlock_rdlock – 읽기-쓰기 잠금을 읽기를 위해 잠금 - pthread_rwlock_tryrdlock – 읽기-쓰기 잠금을 읽기를 위해 잠그지만, 즉시 반환 - pthread_rwlock_wrlock – 읽기-쓰기 잠금을 쓰기를 위해 잠금 - pthread_rwlock_trywrlock – 읽기-쓰기 잠금을 쓰기를 위해 잠그지만, 즉시 반환 - pthread_rwlock_unlock – 읽기-쓰기 잠금을 해제
변환값	성공적 수행: 0, 그렇지 않을 경우: 0이 아닌값

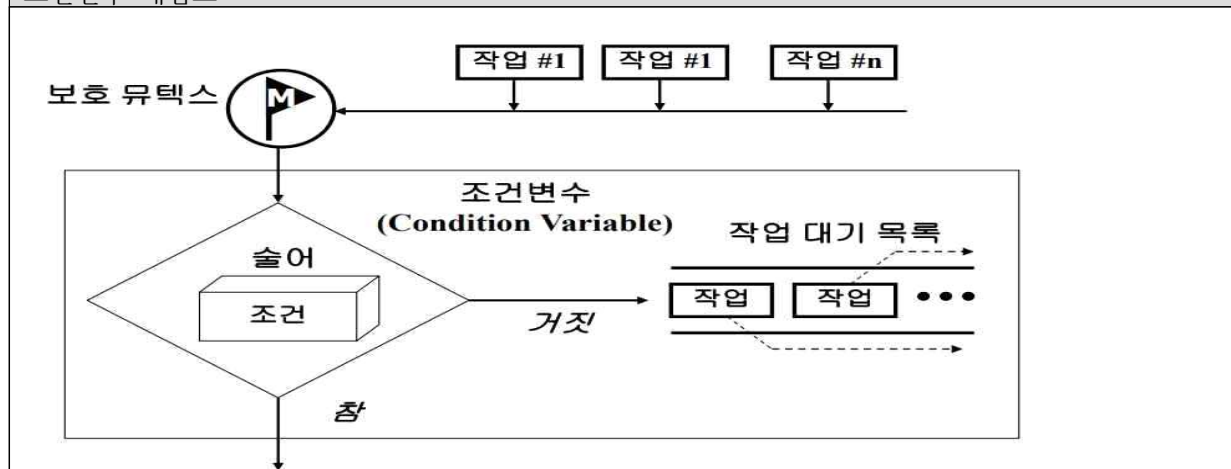
2.8 조건 변수

공유 자원에 접근할 때, 특정한 조건이 만족할 때에만 접근 가능하고 그렇지 않을 때는 대기하도록 하는 동기화 메커니즘

구성요소

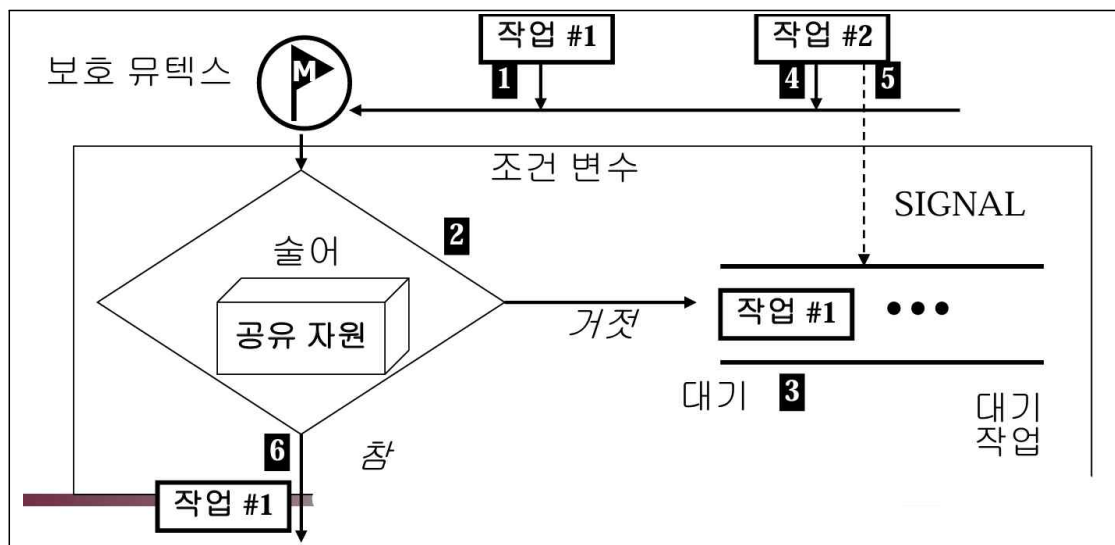
- 술어
 - 공유자원의 조건과 연관된 논리 표현식-참(true)또는 거짓(false)으로 평가
 - 참이면 조건이 만족되어 스레드는 실행 계속
 - 거짓이면 다른 스레드가 조건에 대한 신호를 보낼 때 까지 대기
- 보호 뮤텝스
 - 조건변수에 대한 배제적 접근 보장
 - 스레드는 술어를 평가하기 전에 먼저 뮤텝스를 획득 하여야함
- 스레드 대기 목록
 - 스레드 재실행 순서는 우선순위 또는 FIFO기반

조건변수 개념도



조건변수 주요 연산

- 생성(Create)
 - 조건변수 생성 및 초기화
- 대기(Wait)
 - 조건변수 대기
 - 조건변수 대기를 위해 스레드는 먼저 보호 뮤텍스를 획득하여야 함
 - 뮤텍스 해제와 스레드 대기를 원소적으로 수행
- 신호 (Signal)
 - 조건변수 신호- 조건변수 신호를 위해 스레드는 먼저 보호 뮤텍스를 획득하여야 함
 - 조건변수에서 대기하는 스레드들 중 하나를 깨움
 - 스레드 재실행과 뮤텍스 재획득을 원소적으로 수행
- 브로드캐스트(Broadcast)
 - 조건변수 대기하는 모든 스레드들 신호
 - 모든 스레드들을 깨우지만, 한 스레드만 뮤텍스를 획득하고 다른 스레드들은 뮤텍스의 스레드 대기 목록에 들어감



조건변수 생성 및 파괴 함수

사용법	<pre>#include <pthread.h> pthread_cond_t cond =PTHREAD_COND_INITIALIZER; int pthread_cond_init(pthread_cond_t *cond, const pthread_condattr_t *condattr); int pthread_cond_destroy(pthread_cond_t *cond);</pre>
기능	- PTHREAD_COND_INITIALIZER -조건변수를 초기화 - pthread_cond_init - 조건변수를 동적으로 생성하고 초기화 - pthread_cond_destroy - 동적으로 초기화한 조건변수를 파괴
변환값	- pthread_cond_init 은 항상 0 을 반환 - pthread_cond_destroy 는 성공적 수행: 0, 그렇지 않을 경우: 0이아닌값

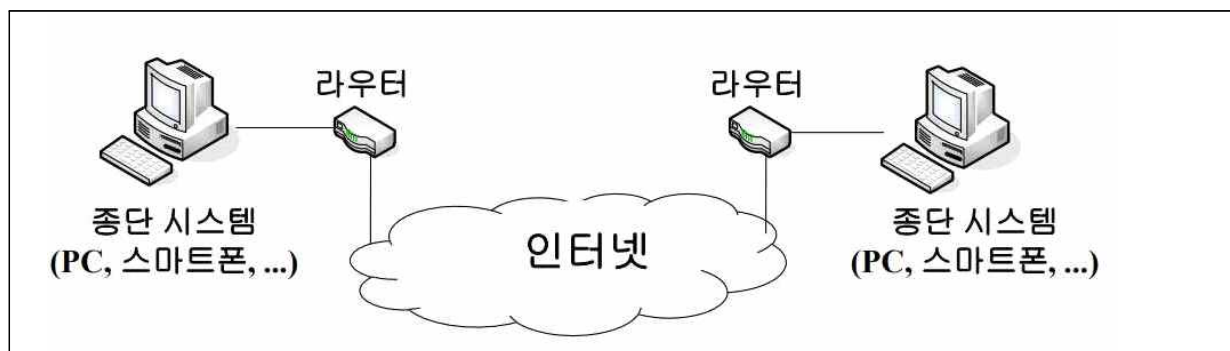
조건변수 대기 함수	
형식	<pre>#include <pthread.h> int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex); int pthread_cond_timedwait(pthread_cond_t *cond, pthread_mutex_t *mutex, const struct timespec *abstime);</pre>
기능	- pthread_cond_wait - 조건변수 상에서 스레드를 대기 - pthread_cond_timedwait - 조건변수 상에서 스레드를 특정시간 동안 대기
변환값	pthread_cond_wait는 항상 0을 반환 pthread_cond_timedwait는 성공적 수행 : 0 ,그렇지않을 경우 : 0이 아닌값

조건변수 신호함수	
형식	<pre>#include <pthread.h> int pthread_cond_signal(pthread_cond_t *cond); int pthread_cond_broadcast(pthread_cond_t *cond);</pre>
기능	- pthread_cond_signal - 조건변수 상에서 신호 하여 대기하는 스레드를 재실행 - pthread_cond_broadcast - 조건변수 상에서 신호하여 대기하는 모든 스레드를 재실행
변환값	항상 0을 반환한다.

3. 소켓 프로그래밍

3.1 인터넷 구성

- 종단 시스템(end-system)
 - 최종 사용자(end-user)를 위한 애플리케이션을 수행하는 주체
- 라우터(router)
 - 종단 시스템이 속한 네트워크와 다른 네트워크를 연결함으로써 서로 다른 네트워크에 속한 종단 시스템끼리 상호 데이터를 교환할 수 있도록 하는 장비

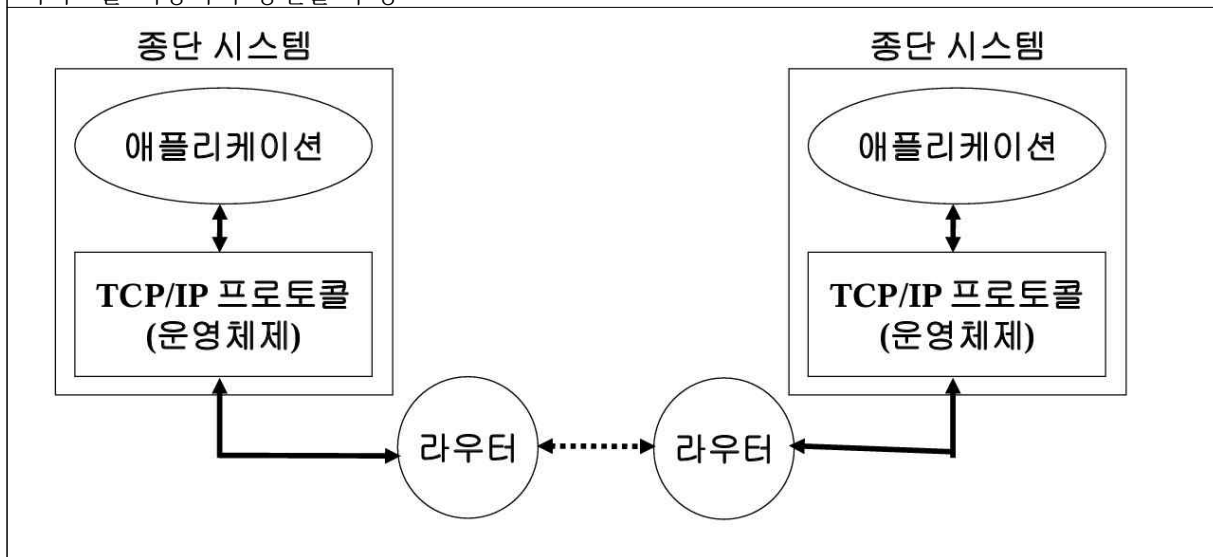


3.2 프로토콜, 패킷

- 프로토콜(protocol)
 - 종단 시스템과 라우터간, 라우터와 라우터간, 그리고 종단 시스템과 종단 시스템 간 통신을 수행하기 위한 정해진 절차와 방법
 - 네트워크 요소들간에 상호 작용을 기술하는 알고리즘과 자료 구조
 - 네트워크 요소들간에 주고 받는 메시지의 형식과 메시지의 수신시 취할 동작 및 에러를 어떻게 다룰 것인가를 기술
- 패킷(packet)
 - 송수신되는 데이터의 블록
 - 링크계층에서는 프레임(frame)이라고도 함
 - SO(International Standards Organization)에서는 PDU(ProtocolData Unit)이라고도 함
 - 상위의 응용계층에서는 메시지(message)라 함

3.2 TCP/IP 프로토콜

- TCP/IP 프로토콜
 - 인터넷의 핵심 프로토콜인 TCP와 IP를 포함한 각종 프로토콜
 - 운영체제에서 구현을 제공하며, 일반 애플리케이션은 운영체제가 제공하는TCP/IP 프로토콜의 서비스를 사용하여 통신을 수행



3.3 소켓의 개요

- 소켓의 정의
 - 응용 프로그램이 데이터를 송수신할 수 있는 소프트웨어적인 추상화 개념
- 소켓의 특징
 - 인터넷의 네트워크간 통신
 - > 같은 컴퓨터의 프로세스끼리 통신 가능
 - > 네트워크에 분산된 서버/클라이언트 시스템 가능
 - 인터넷 주소, 종단 간 프로토콜, 포트번호로 구성
 - 소켓 생성시 프로토콜 종류만 결정
 - 한 소켓의 여러 응용 프로그램에 의해 참조 가능

소켓 생성	
형식	<pre>#include <sys/types.h> #include <sys/socket.h> int socket(int protocolFamily, int type, int protocol);</pre>
기능	- 임의 소켓을 생성하고 소켓 식별자를 반환한다
변환값	성공적인 호출인 경우 소켓 식별자를 반환하고, 수행이 실패한 경우, -1 값을 반환

소켓 해지	
형식	<pre>#include <sys/types.h> #include <sys/socket.h> int close(int socket);</pre>
기능	- 해당 소켓을 해제한다
변환값	성공적인 호출인 경우 소켓 식별자를 반환하고, 수행이 실패한 경우, -1 값을 반환

소켓 주소

- 주소 지정
 - 네트워크를 통한 통신을 위해 특정 호스트접근을 위한 IP 주소와 각 호스트내의 응용을 구분할 수 있는 포트번호 필요

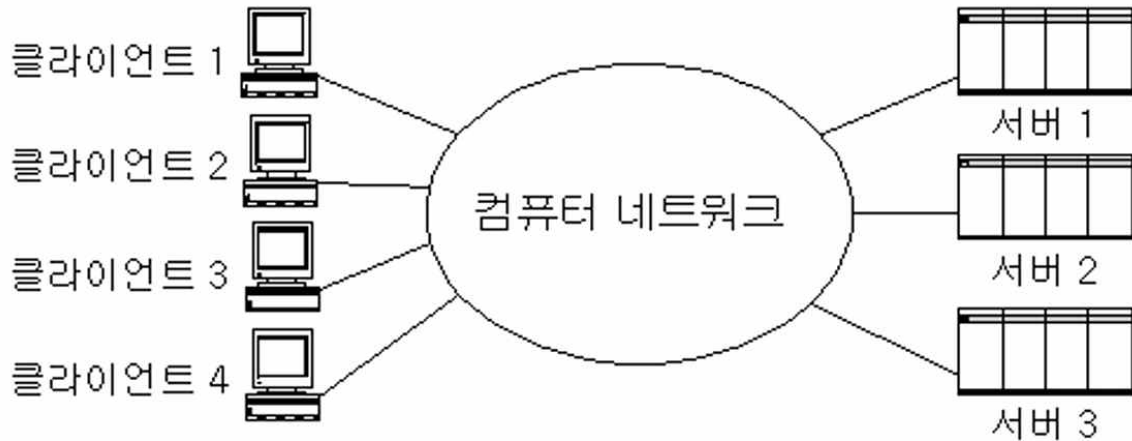
소켓 주소 구조체	
<pre>struct sockaddr { unsigned short sa_family; /* Address family (e.q. AF_INET) */ char sa_data[14]; };</pre>	

TCP/IP 소켓 주소 구조체

```
struct in_addr {
    unsigned long s_addr; /* Internet address (32 bits) */
};
struct sockaddr_in {
    unsigned short sin_family; /* Internet protocol (AF_INET) */
    unsigned short sin_port; /* Address port (16 bits) */
    struct in_addr sin_addr; /* Internet address (32 bits) */
    char sin_zero[8];
};
```

3.4 서버-클라이언트 통신 모델

- 서버
 - 통신 서비스를 제공하는 장비 및 프로그램
- 클라이언트
 - 통신 서비스를 이용하는 장비 및 프로그램



TCP 클라이언트 프로그램

- 클라이언트와 서버의 구분
 - 통신의 임의 단계에서 소켓 인터페이스 사용 방식이 다름
 - 서버는 클라이언트와 접속 되기를 수동적으로 기다림
 - 클라이언트는 능동적으로 서버와 접속하고 통신 시도
- TCP 클라이언트 작업 절차
 - 클라이언트 소켓 생성 - socket()
 - 서버 주소 설정 - 서버 연결에 필요한 구조체 초기화
 - 서버 연결 - connect()
 - 메시지 송수신 - 서버와 메시지 주고 받음, send() & recv()
 - 연결 종료 - close()

소켓 연결	
형식	<pre>#include <sys/types.h> #include <sys/socket.h> int connect(int socket, struct sockaddr *foreignAddress, unsigned int addrlen);</pre>
기능	해당 소켓에 연결요청을 한다.
변환값	성공적인 호출인 경우 0을 반환하고, 수행이 실패한 경우, -1 값을 반환한다.

-

소켓 데이터 송신	
형식	<pre>#include <sys/types.h> #include <sys/socket.h> int send(int socket, const void *msg, unsigned int msgLength, int flags);</pre>
기능	데이터를 해당 소켓을 통해 송신한다.
변환값	성공적인 호출인 경우 0을 반환하고, 수행이 실패한 경우, -1 값을 반환한다.

소켓 데이터 수신	
형식	<pre>#include <sys/types.h> #include <sys/socket.h> int send(int socket, const void *msg, unsigned int msgLength, int flags);</pre>
기능	해당 소켓을 통해 데이터를 수신한다
변환값	성공적인 호출인 경우에는 수신한 바이트수를, 수행이 실패한 경우, -1 값을 서버가 종료되었을 경우에는 0을 반환한다.

3.6 UDP 소켓

TCP와 달리 연결 설정 과정(Handshake) 없이 데이터를 송수신하는 비연결형(Connectionless) 통신 방식 데이터의 신뢰정보보다는 실시간성이나 단순성이 중요한 경우 사용됨

- TCP와의 차이점
 - 메시지 경계 유지: 데이터가 섞이지 않고 보낸 단위대로 수신되므로 별도의 경계 처리가 필요 없음
 - 비신뢰성: 전송된 메시지가 목적지에 도착한다는 보장이 없음 (Best Effort 방식)

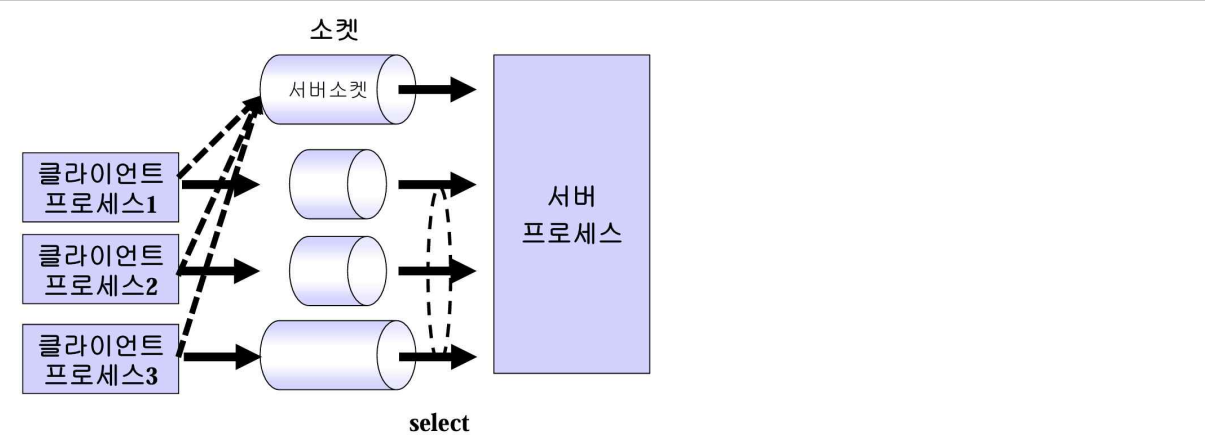
UDP 메시지 송수신 함수	
형식	<pre>#include <sys/types.h> #include <sys/socket.h> int sendto(int socket, const void *msg, unsigned int msgLength, int flags, struct sockaddr *destAddr, unsigned int addrLength); int recvfrom(int socket, void *msg, unsigned int msgLength, int flags, struct sockaddr *srcAddr, unsigned int *addrLength);</pre>
기능	- sendto - 해당 소켓을 통해 목적지(destAddr)로 데이터를 송신 - recvfrom - 해당 소켓을 통해 발신지(srcAddr)로부터 데이터를 수신
변환값	성공 시 전송/수신한 바이트 수(0 포함), 실패 시 -1 반환

3.7 년블로킹 소켓
<블로킹 모드> - 함수 호출 시 관련 동작이 준비되지 않을 경우 프로그램이 진행되지 않고 블록됨 - 블로킹 조건 · recv() 함수 : 수신 버퍼에 데이터가 없을 경우 · send() 함수 : 송신 버퍼가 꽉 차 있어서 데이터를 쓸 수 없는 경우 · connect() 함수 : 서버와 연결 설정이 안 된 경우 · accept() 함수 : 클라이언트와 연결될 때까지 대기 <년블로킹 모드> - 함수 호출 시 관련 동작이 준비되어 있지 않으면 오류를 반환하고 계속 진행 - 주기적으로 상태를 확인하며 반복 실행해야 함 - 비동기 I/O(SIGIO 시그널 이용)를 사용하여 소켓 관련 입출력 사건 발생 시 데이터 처리를 수행하면 효율적임

년블로킹 모드 설정 (파일 제어함수)	
형식	<pre>#include <unistd.h> #include <fcntl.h> int fcntl(int socket, int command, long argument);</pre>
기능	해당 소켓의 제어를 얻어오거나 지정한다
변환값	성공적인 호출인 경우 0 이상의 값을 반환하고, 수행이 실패한 경우, -1값을 반환한다.

3.8 다중 접속 처리
여러 클라이언트가 동시다발적으로 접속하는 서버 멀티플렉싱 - 매번 요청을 처리할 프로세스/스레드 생성이 필요 없음(aio 제외) - select()를 이용한 방법: 반복적으로 파일 디스크립터들(소켓 포함)을 관찰 - epoll을 이용한 방법: · 반복 관찰이 필요 없음 · epoll_create() 함수 : epoll 인스턴스 생성 · epoll_ctl() 함수 : 특정 파일 디스크립터 추가, 삭제, 변경 · epoll_wait() 함수 : 이벤트가 발생한 파일 디스크립터 반환 - aio를 이용한 방법: · 비동기/비봉쇄 I/O 가능하지만, 비동기 처리 시 내부적으로 스레드가 생성되어 비효율적 · lio_listio() 함수 : 다중 입출력 처리

select 를 이용한 소켓프로그래밍

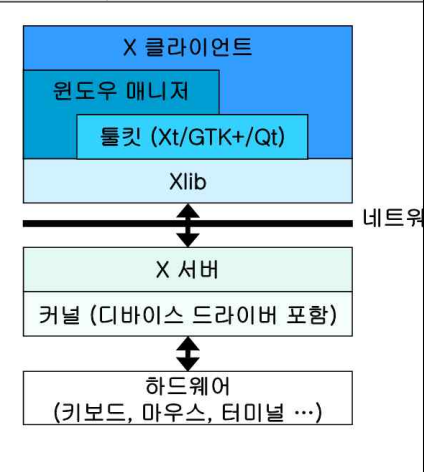


4. 리눅스에서 사용하는 GUI 툴킷

4.1 GUI System Model

- 단일 프로세스 모델
 - 한 프로세스가 그래픽 장치를 전용해서 사용
 - 속도가 빠르지만, 다양한 GUI 프로그램들을 실행하기 어려움
 - GtkFB
- 서버-클라이언트 모델
 1. 서버가 그래픽 장치를 전용해서 사용하는 경우
 - 클라이언트로부터의 그래픽 요청을 처리
 - 안정된 처리를 보장하지만 구조가 복잡하고 크기가 큼(수 MB 이상)
 - X-Windows, Microwindows
 2. 클라이언트와 그래픽 장치를 공유해서 사용하는 경우
 - 서버는 그래픽 장치 사용을 허가
 - 클라이언트가 그래픽 처리를 책임져야 함
 - Qtopia

X 윈도우 개요



X Windows 개요

- 서버-클라이언트 구조
 - 서버가 실제 그래픽 처리를 담당
 - 클라이언트는 입력(키보드, 마우스)에 대한 정보를 요청하고, 터미널 출력을 서버로 보내 터미널 변경을 요구함
- 계층적 구조
 - Xlib Library
 1. 윈도우 생성, 이동, 크기 조정, 폰트, 커서, 도형 관련 작업 수행
 2. 직접 X 프로토콜을 생성하여 서버로 전송
- X Toolkit
 1. 버튼, 풀다운 메뉴 등 고급 GUI 프로그래밍 요소 제공
 2. X 서버와 클라이언트 간의 통신 및 Widget(위젯) GUI 객체 처리
 3. Adena(MIT), OpenLook(AT&T), Motif(OSF), GTK+(GNU), Qt(Trolltech) 등 다양한 위젯 집합 지원
- Window Manager
 1. 데스크톱 화면을 나누어 워크스페이스를 늘리거나, 각 윈도우의 프레임, 최소화, 최대화, 닫기 아이콘, 제목 표시줄, 이동 등의 기능 제공
 2. 또 하나의 X Client 중 하나로 동작

X Windows 개요

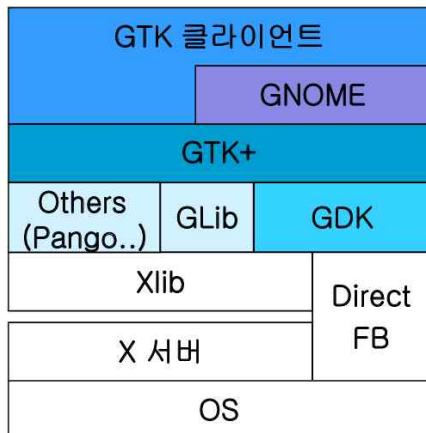
- 장점
 - 네트워크 투명성(Network Transparency)
 - > 네트워크가 투명 하여 사용자 입장에서는 없는 것처럼 보임
 - 모듈성(Modularity) 및 확장성(Extensibility)
 - > 서버-클라이언트구조로 장치 독립성이 보장
 - > 많은 윈도우 자원들이 정해지지 않아 다양한 인터페이스로 적용 가능
 - 오픈소스(Open Source)
 - > 소스가공개
- 단점
 - 상당히 느림
 - 프로그래머에게 상당한 부담
 - 표준 툴킷이 없음
 - 코드가 지나치게 복잡하고 오래 되었음

4.2 Qt

- 멀티플랫폼 GUI 응용을 위한 C++ GUI Toolkit
 - 객체지향 프로그래밍 구현 - 시그널과 슬롯 등 제공
 - 풍부한 위젯(Widget) 제공
 - 다양한 플랫폼 지원-Windows, Mac OS X, Linux, Solaris,HP-UX
 - Unicode 와i18n 등의 다양한 언어 지원
- 유용한 도구 제공
 - QtDesigner(GUI 빌더) , Qt Linguist, qmake, Qt Creator(IDE) 등
- 대표적인 리눅스 데스크탑 환경인 KDE의 기반
 - Qt/X11 사용
- 노르웨이의 Trolltech사에서 개발
- 다양한 배포판으로 제공됨

4.3 GTK+

- 원래GIMP 툴을 위해 개발된 GUI toolkit
 - Qt와 함께 가장 많이 사용하는 오픈소스 그래픽 라이브러리



구성 라이브러리

Glib - 기본 데이터 타입과 오류처리, 메모리 관리 함수들
 GDK -플랫폼의그래픽API 와GTK+ 사이에 위치
 GNOME -다양한 고급GUI 객체(파일선택창등) 제공
 Cairo (Xr/Xc) - 벡터 그래픽스 라이브러리
 Pango-다중 언어를 위한 텍스트와 폰트지원
 libpng, libjpeg, libtiff- 이미지 처리 지원
 FreeType- Truetype 및Type1 폰트 지원 라이브러리

4.4 GLib
데이터 형식, 함수, 메모리 관리, 공통 작업을 처리하는 이식 가능한 C 라이브러리
특징
• 여러 플랫폼에서 개발 가능한 객체형식 지원 • 데이터 형식은 표준 C형식을 대체 • 다양한 데이터 구조 지원 • 동적 로딩을 위한 모듈 지원 • 쓰레드 지원 • 입출력 채널 지원

4.5 Signal 과 Callback
<GTK+ 는 이벤트 기반 라이브러리>
• 사용자가 키보드나 마우스를 통해 입력할 때마다 이벤트 발생 • 이벤트가 발생하면 적절한 함수로 처리하고, 이벤트가 발생하지 않으면 계속 대기
<Signal 과 Callback 매커니즘으로 이벤트 처리>
• 어떤 위젯에 이벤트가 발생하면, 위젯은 시그널을 발생하고 그 시그널과 연결된 콜백 함수 호출 • 사용 방법 - 미리 시그널을 처리할 함수를 만들고 g_signal_connect() 함수를 이용하여 시그널과 처리 함수를 연결

GTK+ Programming
• 기본GTK+ 프로그램의 7 단계 1. 환경 초기화 2. 위젯 생성 및 속성 설정 3. 콜백 루틴 등록 4. 인스턴스 계정 정의 5. 위젯을 화면에 보이게 함 6. 시그널과 이벤트 처리 7. 종료

환경 초기화 함수 gk_init()	
형식	void gtk_init(int *argc_addr, char **argv_addr)
기능	GTK+ 환경을 초기화 한다.
반환값	없음

위젯 생성 함수 gtk_{위젯}_new()	
형식	GtkWidget *gtk_{위젯}_new{꼬리옵션}({인자});
기능	위젯을 생성한다
반환값	생성된 위젯 객체의 포인터를 반환

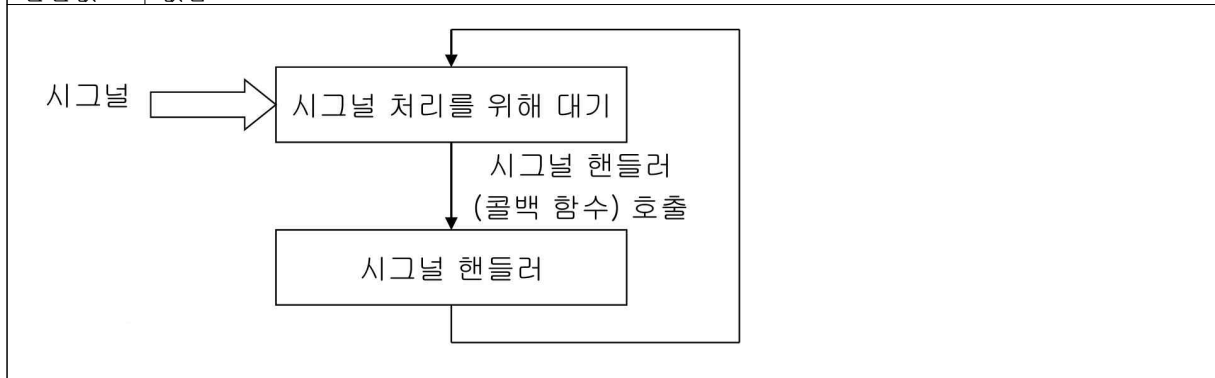
위젯 속성 설정	
형식	void gtk_container_set_border_width(GtkContainer *container, guint width)
기능	위젯의 속성을 설정한다
반환값	없음

콜백 함수 등록	
형식	void g_signal_connect (gpointer instance, gchar *name, GCallback call_routine,gpointer data);
기능	콜백 함수를 등록한다
반환값	없음

인스턴스 계층 구성	
형식	void gtk_container_add(GtkContainer *container, GtkWidget *child) void gtk_box_pack_start(GtkBox *box, GtkWidget *child, gboolean expand, gboolean fill, guint padding)
기능	gtk_Container_add 함수는 child 위젯을 container 위젯에 포함시킨다.
반환값	없음

위젯 화면 표시	
형식	void gtk_widget_show(GtkWidget *widget); void gtk_widget_show_all(GtkWidget *pwidget);
기능	gtk_widget_show 함수는 위젯을 화면에 나타낸다
반환값	없음

시그널과 이벤트 처리	
형식	void gtk_main();
기능	메인 루프를 돌면서 이벤트 및 시그널을 탐지하고 콜백 루틴을 호출하기를 반복한다
반환값	없음

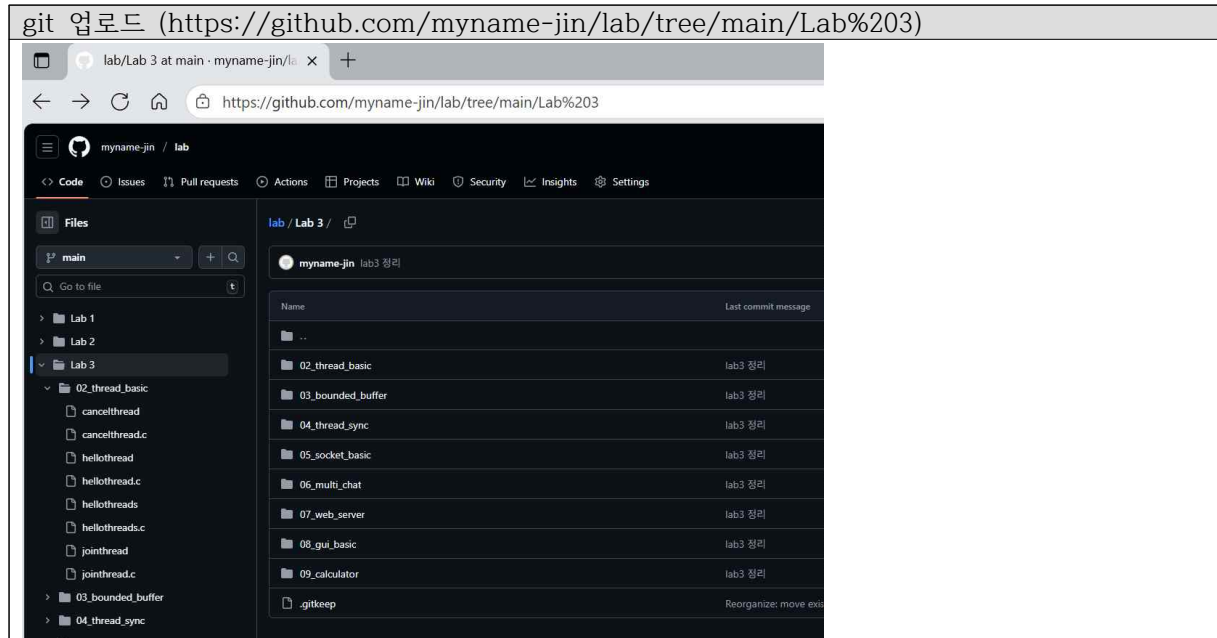


프로그램 종료 함수	
형식	void gtk_main_quit();
기능	GTK+ 프로그램을 종료한다
반환값	없음

Qt Designer 를 활용한 개발	
• Designer를 사용하여 인터페이스 제작 • uic를 사용하여 소스 코드 빌드 • main.cpp 등 소스코드를 필요에 알맞게 편집 • qmake를 사용하여 프로젝트 파일 및 Makefile 생성 • make로 소스파일 컴파일 및 실행 파일 생성	

<실습>

1. 자신의 github 저장소에 lab3 프로젝트를 생성하고 아래의 모든 과제 프로그램을 업로드한다.



2. 스레드 관련 함수들을 사용하여 프로그램을 작성하고 실행하여 보고, 익숙해지도록 사용해 본다.

2.1 스레드 생성과 종료

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

// 스레드 함수
void *hello_thread(void *arg) {
    printf("Thread: Hello, World!\n");
    return arg;
}

int main() {
    pthread_t tid;
    int status;

    // 스레드 생성
    status = pthread_create(&tid, NULL, hello_thread, NULL);
    if (status != 0) {
        perror("Create thread");
        exit(1);
    }

    // 메인 스레드 종료 (생성된 스레드가 실행될 시간을 벌어줌)
    pthread_exit(NULL);
}
```



```
jinho@localhost:~/lab3/02_thread_basic$ vi hellothread.c
jinho@localhost:~/lab3/02_thread_basic$ gcc -o hellothread hellothread.c -pthread
jinho@localhost:~/lab3/02_thread_basic$ ./hellothread
```

-pthread 옵션은 POSIX 쓰레드 라이브러리를 링크(Link)할 뿐만 아니라, 표준 라이브러리가 멀티쓰레드 환경에서 안전하게 동작하도록 전처리기 매크로(Thread-Safe)를 설정해주기 때문에 필수적입니다

실행

```
jinho@localhost:~/lab3/02_thread_basic$ ./hellothread
Thread: Hello, World!
```

2.2 쓰레드 인자 전달

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>

#define NUM_THREADS 3

void *hello_thread(void *arg) {
    // void 포인터를 int로 변환하여 출력
    printf("Thread %d: Hello, World!\n", (int)(long)arg);
    return arg;
}

int main() {
    pthread_t tid[NUM_THREADS];
    int i, status;

    // 3개의 쓰레드 생성
    for (i = 0; i < NUM_THREADS; i++) {
        status = pthread_create(&tid[i], NULL, hello_thread, (void *)i);
        if (status != 0) {
            fprintf(stderr, "Create thread %d: %d", i, status);
            exit(1);
        }
    }

    pthread_exit(NULL);
}
```

컴파일 및 실행

```
jinho@localhost:~/lab3/02_thread_basic$ vi hellothreads.c
jinho@localhost:~/lab3/02_thread_basic$ gcc -o hellothreads hellothreads.c -pthread
jinho@localhost:~/lab3/02_thread_basic$ ./hellothreads
Thread 0: Hello, World!
Thread 1: Hello, World!
Thread 2: Hello, World!
jinho@localhost:~/lab3/02_thread_basic$
```

2.3 쓰레드 결합

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>

void *join_thread(void *arg) {
    int ret = (int)(long)arg; // 인자로 받은 값을 반환값으로 사용
    pthread_exit((void *) (long)ret);
}

int main(int argc, char *argv[]) {
    pthread_t tid;
    int arg, status;
    void *result;

    if (argc < 2) {
        fprintf(stderr, "Usage: jointhread <number>\n");
        exit(1);
    }
    arg = atoi(argv[1]);
    // 쓰레드 생성
    status = pthread_create(&tid, NULL, join_thread, (void *) (long)arg);
    if (status != 0) {
        fprintf(stderr, "Create thread: %d", status);
        exit(1);
    }
    // 쓰레드 종료 대기 (Join)
    status = pthread_join(tid, &result);
    if (status != 0) {
        fprintf(stderr, "Join thread: %d", status);
        exit(1);
    }
    // 결과 확인 (반환값은 쉘의 종료 코드로 사용됨)
    return (int)(long)result;
}
```

컴파일 및 실행

```
jinho@localhost:~/lab3/02_thread_basic$ gcc -o jointhread jointhread.c -pthread
jinho@localhost:~/lab3/02_thread_basic$ ./jointhread 10
jinho@localhost:~/lab3/02_thread_basic$
jinho@localhost:~/lab3/02_thread_basic$ echo $?
10
```

-> 인자로 10을 준다.

2.4 쓰레드 취소

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>

void *cancel_thread(void *arg) {
    int i, state;
    for (i = 0;; i++) {
        // 취소 불가능 상태 설정
        pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, &state);
        printf("Thread: cancel state disabled\n");
        sleep(1);
        // 취소 가능 상태 복구
        pthread_setcancelstate(state, &state);
        printf("Thread: cancel state restored\n");
        // 취소 지점(Cancellation Point) 생성을 위해 sleep 또는 testcancel 호출
        if (i % 5 == 0) pthread_testcancel();
    }
    return arg;
}

int main(int argc, char *argv[]) {
    pthread_t tid;
    int arg, status;
    void *result;

    if (argc < 2) {
        fprintf(stderr, "Usage: cancelthread time(sec)\n");
        exit(1);
    }
    // 쓰레드 생성
    status = pthread_create(&tid, NULL, cancel_thread, NULL);
    if (status != 0) {
        fprintf(stderr, "Create thread: %d", status);
        exit(1);
    }
    // 입력받은 시간만큼 대기 후 취소 요청
    arg = atoi(argv[1]);
    sleep(arg);
    status = pthread_cancel(tid);
    if (status != 0) {
        fprintf(stderr, "Cancel thread: %d", status);
        exit(1);
    }
    // 쓰레드 종료 대기
    status = pthread_join(tid, &result);
    if (status != 0) {
        fprintf(stderr, "Join thread: %d", status);
        exit(1);
    }
}
```

```
// 취소된 스레드의 반환값 확인
return (int)(long)result;
}
```

컴파일 및 실행

```
jinho@localhost:~/lab3/02_thread_basic$ vi cancelthread.c
jinho@localhost:~/lab3/02_thread_basic$ gcc -o cancelthread cancelthread.c -pthread
jinho@localhost:~/lab3/02_thread_basic$ ./cancelthread 2
Thread: cancel state disabled
Thread: cancel state restored
Thread: cancel state disabled
Thread: cancel state restored
jinho@localhost:~/lab3/02_thread_basic$ |
```

-> 인자값을 2를 줬기 때문에 2초후에 종료되는 것을 알 수 있다.

3 스레드를 사용하여 생산자 소비자 문제를 해결하는 제한버퍼를 생성하고 활용하는 프로그램을 구현하시오. 단, 생산자와 소비자 스레드는 각각 둘 이상 가능해야 한다.

코드

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define MAX 5      // 버퍼 크기
#define ITEMS 10   // 처리 아이템 수

int buffer[MAX];
int in = 0, out = 0, count = 0; // 공유 자원

// 정적 초기화
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;
pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;

void *producer(void *arg) {
    long id = (long)arg;
    for (int i = 0; i < ITEMS; i++) {
        usleep(rand() % 10000); // 생산 시간 시뮬레이션

        pthread_mutex_lock(&mutex);
        while (count == MAX) { // 버퍼가 꽉 찼으면 대기
            printf("[Producer %ld] Buffer Full. Waiting...\n", id);
            pthread_cond_wait(&not_full, &mutex);
        }

        buffer[in] = rand() % 100;
        in = (in + 1) % MAX;
        count++;
        printf("[Producer %ld] Inserted: %d (Count: %d)\n", id, buffer[(in-1+MAX)%MAX], count);
    }
}
```

```

        pthread_cond_signal(&not_empty); // 비어있지 않다고 알림
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

void *consumer(void *arg) {
    long id = (long)arg;
    for (int i = 0; i < ITEMS; i++) {
        pthread_mutex_lock(&mutex);
        while (count == 0) { // 버퍼가 비었으면 대기
            printf("\t<Consumer %ld> Buffer Empty. Waiting...\n", id);
            pthread_cond_wait(&not_empty, &mutex);
        }

        int item = buffer[out];
        out = (out + 1) % MAX;
        count--;
        printf("\t<Consumer %ld> Removed: %d (Count: %d)\n", id, item, count);

        pthread_cond_signal(&not_full); // 꼭 차지 않았다고 알림
        pthread_mutex_unlock(&mutex);
        usleep(rand() % 10000); // 소비 시간 시뮬레이션
    }
    return NULL;
}

int main() {
    pthread_t t[4]; // 0,1: Producer / 2,3: Consumer
    srand(time(NULL));

    // 쓰레드 생성 (생산자 2명, 소비자 2명)
    for (long i = 0; i < 4; i++) {
        if (i < 2) pthread_create(&t[i], NULL, producer, (void*)i);
        else      pthread_create(&t[i], NULL, consumer, (void*)i);
    }

    // 쓰레드 대기
    for (int i = 0; i < 4; i++) pthread_join(t[i], NULL);

    printf("All threads finished.\n");
    return 0;
}

```

컴파일 및 실행

```
jinho@localhost:~/lab3/02_thread_basic$ vi prod_cons.c
jinho@localhost:~/lab3/02_thread_basic$ gcc -o prod_cons prod_cons.c -pthread
jinho@localhost:~/lab3/02_thread_basic$ ./prod_cons
<Consumer 2> Buffer Empty. Waiting...
<Consumer 3> Buffer Empty. Waiting...
[Producer 0] Inserted: 13 (Count: 1)
<Consumer 2> Removed: 13 (Count: 0)
<Consumer 2> Buffer Empty. Waiting...
[Producer 1] Inserted: 78 (Count: 1)
<Consumer 3> Removed: 78 (Count: 0)
[Producer 0] Inserted: 67 (Count: 1)
<Consumer 2> Removed: 67 (Count: 0)
<Consumer 3> Buffer Empty. Waiting...
[Producer 0] Inserted: 75 (Count: 1)
<Consumer 3> Removed: 75 (Count: 0)
[Producer 1] Inserted: 1 (Count: 1)
[Producer 0] Inserted: 96 (Count: 2)
<Consumer 2> Removed: 1 (Count: 1)
<Consumer 3> Removed: 96 (Count: 0)
[Producer 1] Inserted: 92 (Count: 1)
<Consumer 3> Removed: 92 (Count: 0)
[Producer 1] Inserted: 20 (Count: 1)
<Consumer 2> Removed: 20 (Count: 0)
[Producer 0] Inserted: 65 (Count: 1)
[Producer 0] Inserted: 13 (Count: 2)
[Producer 1] Inserted: 9 (Count: 3)
[Producer 1] Inserted: 99 (Count: 4)
<Consumer 3> Removed: 65 (Count: 3)
[Producer 0] Inserted: 13 (Count: 4)
[Producer 0] Inserted: 83 (Count: 5)
<Consumer 2> Removed: 13 (Count: 4)
[Producer 1] Inserted: 86 (Count: 5)
[Producer 1] Buffer Full. Waiting...
[Producer 0] Buffer Full. Waiting...
<Consumer 2> Removed: 9 (Count: 4)
[Producer 1] Inserted: 71 (Count: 5)
<Consumer 3> Removed: 99 (Count: 4)
[Producer 0] Inserted: 11 (Count: 5)
<Consumer 3> Removed: 13 (Count: 4)
[Producer 0] Inserted: 66 (Count: 5)
[Producer 1] Buffer Full. Waiting...
<Consumer 3> Removed: 83 (Count: 4)
[Producer 1] Inserted: 39 (Count: 5)
<Consumer 2> Removed: 86 (Count: 4)
[Producer 1] Inserted: 59 (Count: 5)
<Consumer 2> Removed: 71 (Count: 4)
<Consumer 3> Removed: 11 (Count: 3)
<Consumer 2> Removed: 66 (Count: 2)
<Consumer 2> Removed: 39 (Count: 1)
<Consumer 3> Removed: 59 (Count: 0)
All threads finished.
jinho@localhost:~/lab3/02_thread_basic$
```

- 초기 동기화 확인: 프로그램 실행 초기, 버퍼가 비어있으므로(Count: 0) 소비자 쓰레드들이 데이터를 가져가지 않고 Waiting... 상태로 대기하며 조건변수가 정상 작동함을 확인하였습니다.
- 생산자-소비자 패턴: 생산자가 데이터를 삽입(Inserted)하여 버퍼를 채우면(Count 증가), 대기 하던 소비자가 깨어나(Signal) 데이터를 제거(Removed)하고 Count를 감소시키는 과정이 반복 됩니다.
- 데이터 무결성 보장: 여러 쓰레드가 동시에 접근함에도 불구하고 Count 변수가 꼬이지 않고 정확하게 증감하며, 먼저 생산된 데이터가 먼저 소비되는(FIFO) 흐름을 통해 뮤텁스(Mutex)를 이용한 상호 배제가 잘 이루어지고 있음을 알 수 있습니다.

4. 부모와 자식 쓰레드가 1초마다 한번씩 번갈아가면서 “hello child/parent” 메시지를 출력하도록 실행하는 쓰레드 프로그램을 작성하시오. 즉, 하나의 이진 플래그를 사용하고 상호 배제 및 동기화를 위해 뮤텁스와 조건변수를 사용한다.

코드

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// 공유 자원 및 동기화 도구
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

// 이진 플래그 (0: 부모 차례, 1: 자식 차례)
int flag = 0;

// 반복 횟수 설정
#define ITERATIONS 5
```

```

void *child_thread(void *arg) {
    for (int i = 0; i < ITERATIONS; i++) {
        pthread_mutex_lock(&mutex);

        // 내 차례(flag == 1)가 아니면 대기
        while (flag != 1) {
            pthread_cond_wait(&cond, &mutex);
        }

        printf("hello child\n");
        sleep(1); // 1초 대기

        // 턴을 부모(0)에게 넘김
        flag = 0;
        pthread_cond_signal(&cond); // 대기 중인 부모 깨우기
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

int main() {
    pthread_t tid;

    // 자식 쓰레드 생성
    if (pthread_create(&tid, NULL, child_thread, NULL) != 0) {
        perror("pthread_create");
        return 1;
    }

    // 부모 쓰레드 로직
    for (int i = 0; i < ITERATIONS; i++) {
        pthread_mutex_lock(&mutex);

        // 내 차례(flag == 0)가 아니면 대기
        while (flag != 0) {
            pthread_cond_wait(&cond, &mutex);
        }

        printf("hello parent\n");
        sleep(1); // 1초 대기

        // 턴을 자식(1)에게 넘김
        flag = 1;
        pthread_cond_signal(&cond); // 대기 중인 자식 깨우기
        pthread_mutex_unlock(&mutex);
    }

    // 자식 쓰레드 종료 대기
    pthread_join(tid, NULL);

    return 0;
}

```

컴파일 및 실행

```
jinho@localhost:~/lab3/04_thread_sync$ gcc -o sync_msg sync_msg.c -pthread
jinho@localhost:~/lab3/04_thread_sync$ ./sync_msg
hello parent
hello child
hello parent
hello child
hello parent
hello child
hello parent
hello child
hello parent
hello child
```

- 실행 결과를 통해 부모와 자식 스레드가 경쟁 없이 정확히 1초 간격으로 번갈아 가며 출력됨을 확인하였습니다. 이는 뮤텍스로 임계 영역을 보호하고, 조건변수와 플래그(flag)를 사용하여 각 스레드의 실행 순서를 엄격하게 제어한 결과입니다."

5. 소켓을 이용하여 프로그램을 작성하고 실행하여 보고, 익숙해지도록 사용해 본다.

5.1 TCP 소켓 (클라이언트 코드)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define BUF_SIZE 1024
#define PORT 9190
#define IP "127.0.0.1"

void error_handling(char *message) {
    perror(message);
    exit(1);
}

int main() {
    int sock;
    struct sockaddr_in serv_addr;
    char message[BUF_SIZE];
    int str_len, recv_len, recv_cnt;

    sock = socket(PF_INET, SOCK_STREAM, 0);
    if (sock == -1) error_handling("socket() error");

    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_addr.s_addr = inet_addr(IP);
    serv_addr.sin_port = htons(PORT);

    if (connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1)
```



```

        error_handling("connect() error");
    else
        printf("Connected to Server! (Type 'Q' to quit)\n");

    while (1) {
        fputs("Input message: ", stdout);
        fgets(message, BUF_SIZE, stdin);

        if (!strcmp(message, "q\n") || !strcmp(message, "Q\n"))
            break;

        str_len = write(sock, message, strlen(message)); // 1. 데이터 전송 및 길이 저장

        recv_len = 0;
        while (recv_len < str_len) {
            recv_cnt = read(sock, &message[recv_len], BUF_SIZE - 1);
            if (recv_cnt == -1) error_handling("read() error");
            recv_len += recv_cnt; // 읽은 만큼 누적
        }

        message[recv_len] = 0; // 문자열 끝 처리
        printf("Message from server: %s", message);
    }

    close(sock);
    return 0;
}

```

클라이언트 실행결과

```

jinho@localhost:~/lab3/05_socket_basic$ ./echo_client
Connected to Server! (Type 'Q' to quit)
Input message: tcp 통신 테스트입니다
Message from server: tcp 통신 테스트입니다
Input message: 20212979 임진호 입니다.
Message from server: 20212979 임진호 입니다.
Input message: q
jinho@localhost:~/lab3/05_socket_basic$ |

```

5.1 TCP 소켓 (서버 코드)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define BUF_SIZE 1024
#define PORT 9190

```

```

void error_handling(char *message) {
    perror(message);
    exit(1);
}

int main() {
    int serv_sock, clnt_sock;
    struct sockaddr_in serv_addr, clnt_addr;
    socklen_t clnt_addr_size;
    char message[BUF_SIZE];
    int str_len;

    // 1. 서버 소켓 생성 (IPv4, TCP 스트림)
    serv_sock = socket(PF_INET, SOCK_STREAM, 0);
    if (serv_sock == -1) error_handling("socket() error");

    // 2. 주소 구조체 초기화 및 설정
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;           // IPv4 주소 체계
    serv_addr.sin_addr.s_addr = htonl(INADDR_ANY); // 현재 시스템의 IP 주소 자동 할당
    serv_addr.sin_port = htons(PORT);        // 포트 번호 할당

    // 3. 소켓에 IP 주소와 포트 번호 할당 (Binding)
    if (bind(serv_sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1)
        error_handling("bind() error");

    // 4. 연결 요청 대기 상태로 진입 (Listening, 대기열 크기: 5)
    if (listen(serv_sock, 5) == -1)
        error_handling("listen() error");

    printf("Server Start! Waiting for connection...\n");

    clnt_addr_size = sizeof(clnt_addr);

    // 5. 클라이언트의 연결 요청 수락 (Blocking 모드)
    // 연결 요청이 들어올 때까지 대기하며, 연결 시 새로운 소켓 디스크립터 반환
    clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_addr, &clnt_addr_size);
    if (clnt_sock == -1) error_handling("accept() error");

    printf("Client Connected!\n");

    // 6. 데이터 송수신 (Echo 서비스 수행)
    // 클라이언트로부터 수신된 데이터를 버퍼에 저장 후 즉시 재전송
    while ((str_len = read(clnt_sock, message, BUF_SIZE)) != 0) {
        write(clnt_sock, message, str_len);
    }

    // 7. 소켓 리소스 해제 및 종료
    close(clnt_sock);
    close(serv_sock);
    return 0;
}

```

서버 실행결과

```
jinho@localhost:~/lab3/05_socket_basic$ ./echo_server
Server Start! Waiting for connection...
```

클라이언트 접속

```
jinho@localhost:~/lab3/05_socket_basic$ ./echo_server
Server Start! Waiting for connection...
Client Connected!
```

5.2 UDP 소켓 (클라이언트 코드)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define BUF_SIZE 1024
#define PORT 9190
#define IP "127.0.0.1"

void error_handling(char *message) {
    perror(message);
    exit(1);
}

int main() {
    int sock;
    struct sockaddr_in serv_addr, from_addr;
    socklen_t adr_sz;
    char message[BUF_SIZE];
    int str_len;

    sock = socket(PF_INET, SOCK_DGRAM, 0);    // 1. UDP 소켓 생성
    if (sock == -1) error_handling("socket() error");

    memset(&serv_addr, 0, sizeof(serv_addr));    // 보낼 곳(서버)의 주소 설정
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_addr.s_addr = inet_addr(IP);
    serv_addr.sin_port = htons(PORT);

    while (1) {
        fputs("Insert message(q to quit): ", stdout);
        fgets(message, sizeof(message), stdin);

        if (!strcmp(message, "q\n") || !strcmp(message, "Q\n"))
            break;

        sendto(sock, message, strlen(message), 0, // 2. 데이터 전송 (sendto)
            (struct sockaddr*)&serv_addr, sizeof(serv_addr));

        adr_sz = sizeof(from_addr);
```

```

// 3. 데이터 수신 (recvfrom)
// 서버가 예코로 돌려보낸 데이터를 받습니다.
str_len = recvfrom(sock, message, BUF_SIZE, 0,
                   (struct sockaddr*)&from_addr, &adr_sz);
message[str_len] = 0;
printf("Message from server: %s", message);
}
close(sock);
return 0;
}

```

클라이언트 실행결과

```

jinho@localhost:~/lab3/05_socket_basic$ ./udp_client
Insert message(q to quit): UDP 소켓통신 테스트
Message from server: UDP 소켓통신 테스트
Insert message(q to quit): 20212979 임진호
Message from server: 20212979 임진호
Insert message(q to quit): 리눅스 실습
Message from server: 리눅스 실습
Insert message(q to quit): Q
jinho@localhost:~/lab3/05_socket_basic$ ./udp_client

```

5.2 UDP 소켓 (서버 코드)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define BUF_SIZE 1024
#define PORT 9190

void error_handling(char *message) {
    perror(message);
    exit(1);
}

int main() {
    int serv_sock;
    struct sockaddr_in serv_addr, clnt_addr;
    socklen_t clnt_addr_size;
    char message[BUF_SIZE];
    int str_len;

    serv_sock = socket(PF_INET, SOCK_DGRAM, 0);          // 1. UDP 소켓 생성 (SOCK_DGRAM)
    if (serv_sock == -1) error_handling("UDP socket creation error");

```

```

memset(&serv_addr, 0, sizeof(serv_addr));      // 2. 주소 설정
serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
serv_addr.sin_port = htons(PORT);

if (bind(serv_sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1)    // 3. 소켓에 주소 할당 (Bind)
    error_handling("bind() error");

printf("UDP Server Start!\n");

while (1) {
    clnt_addr_size = sizeof(clnt_addr);

    str_len = recvfrom(serv_sock, message, BUF_SIZE, 0,          // 4. 데이터 수신 (recvfrom)
                       (struct sockaddr*)&clnt_addr, &clnt_addr_size);

    sendto(serv_sock, message, str_len, 0,                      // 5. 데이터 송신 (sendto)
            (struct sockaddr*)&clnt_addr, clnt_addr_size);
}

close(serv_sock);
return 0;
}

```

서버 실행결과

```

jinho@localhost:~/lab3/05_socket_basic$ ./udp_server
UDP Server Start!

```

-> UDP소켓은 비연결성 프로토콜이기 때문에, 서버가 클라이언트의 접속 상태를 별도로 추적하지 않습니다. 따라서 접속 성공 메시지 없이 오직 수신된 데이터그램 패킷만을 독립적으로 처리하는 UDP의 특징을 확인하였다.

6. 멀티프로세스/쓰레드, select 또는 epoll 을 사용하여 다중 클라이언트를 처리하는 채팅 프로그램을 구현하시오.

select를 이용한 채팅 프로그램 클라이언트

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/select.h>

#define BUF_SIZE 1024
#define PORT 9190
#define IP "127.0.0.1"

void error_handling(char *buf) { perror(buf); exit(1); }

```

```

int main(int argc, char *argv[]) {
    int sock, str_len;
    char buf[BUF_SIZE], name[BUF_SIZE], msg[BUF_SIZE];
    struct sockaddr_in serv_adr;
    fd_set reads, cpy_reads;

    if (argc != 2) {    // 실행 시 이름 입력
        printf("Usage : %s <Name>\n", argv[0]);
        exit(1);
    }
    sprintf(name, "[%s]", argv[1]);

    sock = socket(PF_INET, SOCK_STREAM, 0);
    memset(&serv_adr, 0, sizeof(serv_adr));
    serv_adr.sin_family = AF_INET;
    serv_adr.sin_addr.s_addr = inet_addr(IP);
    serv_adr.sin_port = htons(PORT);

    if (connect(sock, (struct sockaddr*)&serv_adr, sizeof(serv_adr)) == -1)
        error_handling("connect() error");

    printf("Chat Connected!\n");

    FD_ZERO(&reads);
    FD_SET(0, &reads);    // 0: 표준 입력(키보드) 감시
    FD_SET(sock, &reads); // sock: 서버 소켓 감시

    while (1) {
        cpy_reads = reads;
        if (select(sock + 1, &cpy_reads, 0, 0, 0) == -1) break;

        if (FD_ISSET(sock, &cpy_reads)) {    // 1. 서버에서 데이터 옴 (수신)
            str_len = read(sock, buf, BUF_SIZE - 1);
            if (str_len == 0) return 0;
            buf[str_len] = 0;
            printf("%s", buf); // 화면 출력
        }
        if (FD_ISSET(0, &cpy_reads)) { // 2. 키보드 입력 있음 (송신)
            str_len = read(0, buf, BUF_SIZE);
            if (str_len == 0) break;
            buf[str_len] = 0;

            sprintf(msg, "%s %s", name, buf); // 이름 붙여서 전송
            write(sock, msg, strlen(msg));
        }
    }
    close(sock);
    return 0;
}

```

select를 이용한 채팅 프로그램 서버

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <sys/select.h>

#define BUF_SIZE 1024
#define PORT 9190

void error_handling(char *buf) { perror(buf); exit(1); }

int main() {
    int serv_sock, clnt_sock, fd_max, str_len, i, j;
    struct sockaddr_in serv_adr, clnt_adr;
    socklen_t adr_sz;
    char buf[BUF_SIZE];
    fd_set reads, cpy_reads;

    serv_sock = socket(PF_INET, SOCK_STREAM, 0);    // 1. 서버 소켓 생성
    memset(&serv_adr, 0, sizeof(serv_adr));
    serv_adr.sin_family = AF_INET;
    serv_adr.sin_addr.s_addr = htonl(INADDR_ANY);
    serv_adr.sin_port = htons(PORT);

    if (bind(serv_sock, (struct sockaddr*)&serv_adr, sizeof(serv_adr)) == -1) error_handling("bind() error");
    if (listen(serv_sock, 5) == -1) error_handling("listen() error");

    FD_ZERO(&reads);    // 2. select 초기화
    FD_SET(serv_sock, &reads); // 서버 소켓 등록 (연결 요청 감시)
    fd_max = serv_sock;

    printf("Chat Server Start!\n");

    while (1) {
        cpy_reads = reads;
        // 3. 변화 감지 (무한 대기)
        if (select(fd_max + 1, &cpy_reads, 0, 0, 0) == -1) break;

        for (i = 0; i < fd_max + 1; i++) {
            if (FD_ISSET(i, &cpy_reads)) { // 변화가 있는 소켓 발견

                // Case A: 연결 요청 (서버 소켓)
                if (i == serv_sock) {
                    adr_sz = sizeof(clnt_adr);
                    clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_adr, &adr_sz);
                    FD_SET(clnt_sock, &reads); // 감시 목록에 추가
                    if (fd_max < clnt_sock) fd_max = clnt_sock;
                    printf("Connected client: %d\n", clnt_sock);
                }
            }
        }
    }
}
```

```

// Case B: 데이터 수신 (클라이언트 소켓)
else {
    str_len = read(i, buf, BUF_SIZE);
    if (str_len == 0) { // 연결 종료
        FD_CLR(i, &reads);
        close(i);
        printf("Closed client: %d\n", i);
    } else {
        // 메시지 방송 (Broadcast): 나(i)를 제외한 모든 접속자에게 전송
        for (j = 0; j < fd_max + 1; j++) {
            if (FD_ISSET(j, &reads) && j != serv_sock && j != i)
                write(j, buf, str_len);
        }
    }
}
}
}
}
close(serv_sock);
return 0;
}

```

실행 결과(서버)

```

jinho@localhost:~/lab3/06_multi_chat$ ./chat_server
Chat Server Start!
Connected client: 4
Connected client: 5
Closed client: 4
Closed client: 5

```

프로그램을 실행하면 Chat Server Start!와 함께 클라이언트를 대기하고 접속하거나 접속을 끊으면 위와 같은 결과가 나온다.

user라는 이름의 클라이언트 실행	Jinho라는 이름의 클라이언트 실행
<pre> jinho@localhost:~/lab3/06_multi_chat\$./chat_client user Chat Connected! [Jinho] 안녕 리눅스 수업은 잘 듣고 있니? 어 잘 듣고 있어 [Jinho] 어려운 내용은 없어? 명령어가 익숙해 저서 어렵지 않아 [Jinho] select로 채팅 프로그램 만들었는데 괜찮니? 오류 없이 시스템이 잘 돌아가고 있어! ^C jinho@localhost:~/lab3/06_multi_chat\$ </pre>	<pre> jinho@localhost:~/lab3/06_multi_chat\$./chat_client Jinho Chat Connected! 안녕 리눅스 수업은 잘 듣고 있니? [user] 어 잘 듣고 있어 어려운 내용은 없어? [user] 명령어가 익숙해 저서 어렵지 않아 select로 채팅 프로그램 만들었는데 괜찮니? [user] 오류 없이 시스템이 잘 돌아가고 있어! ^C jinho@localhost:~/lab3/06_multi_chat\$ </pre>

7. TCP 소켓을 이용하여 HTTP GET 및 POST 메소드 및 CGI 프로그램 실행을 구현하는 간단한 웹서버를 구현하시오.

웹서버 코드

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/wait.h>

#define BUF_SIZE 1024
#define PORT 8080

void error_handling(char *buf) { perror(buf); exit(1); }

// 정적 파일(HTML) 전송 함수 (GET 요청)
void send_file(int clnt_sock, char *file_name) {
    char buf[BUF_SIZE];
    FILE *fp;

    fp = fopen(file_name, "r");
    if (fp == NULL) {
        write(clnt_sock, "HTTP/1.1 404 Not Found\r\n\r\n", 26);
        return;
    }

    char header[] = "HTTP/1.1 200 OK\r\nContent-Type: text/html; charset=UTF-8\r\n\r\n";
    write(clnt_sock, header, strlen(header));

    while (fgets(buf, BUF_SIZE, fp) != NULL) {
        write(clnt_sock, buf, strlen(buf));
    }
    fclose(fp);
}

// CGI 실행 함수 (POST 요청) - 파이프 사용
// read_data: 부모가 소켓에서 이미 읽은 데이터 (POST Body)
void execute_cgi(int clnt_sock, char *path, char *method, char *read_data) {
    char header[] = "HTTP/1.1 200 OK\r\nContent-Type: text/html; charset=UTF-8\r\n\r\n";
    write(clnt_sock, header, strlen(header));

    int fds[2]; // 파이프 생성 (0: 읽기용, 1: 쓰기용)
    if (pipe(fds) == -1) {
        perror("pipe error");
        return;
    }

    int pid = fork();
```

```

if (pid == 0) { // [자식 프로세스: 파이썬 실행]
    close(fds[1]); // 자식은 쓰기 안 함

    // 1. 파이프의 읽기 구멍(fds[0])을 표준 입력(STDIN)으로 연결
    // -> 부모가 파이프에 넣어준 데이터를 sys.stdin.read()로 읽을 수 있게 됨
    dup2(fds[0], STDIN_FILENO);

    // 2. 소켓을 표준 출력(STDOUT)으로 연결
    // -> print() 하면 클라이언트에게 날아감
    dup2(clnt_sock, STDOUT_FILENO);

    // 3. 파이썬 실행
    execl("/usr/bin/python3", "python3", path, NULL);
    exit(0);
} else { // [부모 프로세스: 데이터 전달]
    close(fds[0]); // 부모는 읽기 안 함

    // 4. 아까 읽어둔 데이터(read_data)를 파이프에 밀어넣음
    write(fds[1], read_data, strlen(read_data));

    // 5. 다 썼으니 파이프 닫음 (자식에게 EOF 전달 -> 자식의 read가 끝남)
    close(fds[1]);

    // 부모 역할 끝. 소켓은 자식이 쓰고 있으니 부모 쪽에서는 닫음
    close(clnt_sock);

    // 자식 프로세스 정리 (좀비 방지)
    waitpid(pid, NULL, WNOHANG);
}
}

int main() {
    int serv_sock, clnt_sock;
    struct sockaddr_in serv_adr, clnt_adr;
    socklen_t adr_sz;
    char buf[BUF_SIZE];
    char method[10], path[100];

    serv_sock = socket(PF_INET, SOCK_STREAM, 0);
    memset(&serv_adr, 0, sizeof(serv_adr));
    serv_adr.sin_family = AF_INET;
    serv_adr.sin_addr.s_addr = htonl(INADDR_ANY);
    serv_adr.sin_port = htons(PORT);

    int opt = 1;
    setsockopt(serv_sock, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    if (bind(serv_sock, (struct sockaddr*)&serv_adr, sizeof(serv_adr)) == -1) error_handling("bind() error");
    if (listen(serv_sock, 5) == -1) error_handling("listen() error");

    printf("Web Server Started on Port %d...\n", PORT);
}

```

```

while (1) {
    adr_sz = sizeof(clnt_adr);
    clnt_sock = accept(serv_sock, (struct sockaddr*)&clnt_adr, &adr_sz);
    if (clnt_sock == -1) continue;

    // 1. 요청 읽기 (여기서 소켓 데이터를 읽어 buf에 저장)
    int len = read(clnt_sock, buf, BUF_SIZE - 1);
    if (len > 0) {
        buf[len] = 0;
        // 요청 메소드와 경로 파싱
        sscanf(buf, "%s %s", method, path);
        printf("Request: %s %s\n", method, path);

        if (strstr(path, ".py")) {
            // [POST] 파이션 파일이면 CGI 실행 (읽은 데이터 buf도 같이 넘김)
            execute_cgi(clnt_sock, path + 1, method, buf);
        } else {
            // [GET] 아니면 파일 전송
            if (strcmp(path, "/") == 0) strcpy(path, "/index.html");
            send_file(clnt_sock, path + 1);
            close(clnt_sock);
        }
    } else {
        close(clnt_sock);
    }
}
close(serv_sock);
return 0;
}

```

웹 페이지

```

<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>C Web Server</title>
</head><body>
    <h1>✖ Simple Calculator (CGI)</h1>
    <p>C언어 소켓 서버에서 Python CGI를 실행합니다.</p>

    <form action="calc.py" method="POST">
        <input type="number" name="num1" placeholder="숫자 1" required>
        X
        <input type="number" name="num2" placeholder="숫자 2" required>
        <button type="submit">계산하기</button>
    </form> </body> </html>

```

CGI 프로그램

```

import sys
import os

# CGI 스크립트: 표준 입력(Stdin)을 통해 POST 데이터를 수신

```

```

try:
    content = sys.stdin.read()    # 1. 표준 입력에서 전체 데이터 읽기

    # 2. HTTP 헤더와 바디 분리
    # 이중 개행문자(\r\n\r\n)를 기준으로 헤더와 바디를 구분
    if "\r\n\r\n" in content:
        body = content.split("\r\n\r\n")[1]
    else:
        body = content

    # 3. 파라미터 파싱 (num1=10&num2=20 형태)
    params = {}
    for pair in body.split('&'):
        if '=' in pair:
            key, value = pair.split('=')
            params[key] = value

    # 4. 데이터 연산 수행
    n1 = int(params.get('num1', 0))
    n2 = int(params.get('num2', 0))
    result = n1 * n2

    # 5. 결과 HTML 생성 및 표준 출력(Stdout)으로 전송
    print(f"<html><body>")
    print(f"<h2>Result: {n1} X {n2} = <span style='color:red'>{result}</span></h2>")
    print(f"<br><a href='/'>Go Back</a>")
    print(f"</body></html>")
except Exception as e:
    # 예외 처리: 에러 메시지 출력
    print(f"<html><body><h2>Error: {e}</h2></body></html>")

```

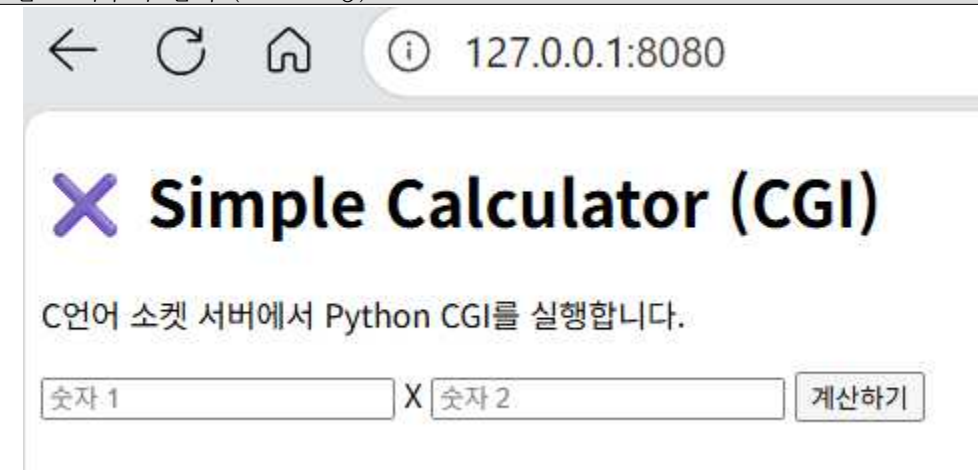
서버 컴파일 및 실행

```

jinho@localhost:~/lab3/07_web_server$ gcc -o http_server http_server.c
jinho@localhost:~/lab3/07_web_server$ ./http_server
Web Server Started on Port 8080...

```

웹 브라우저 접속 (GET 요청)



-> <http://127.0.0.1:8080> URL을 입력하면 위와 같은 index.html에 코딩한 Simple Calculator가 나온다 또한 서버에 아래와 같은 값이 출력된다

```
jinho@localhost:~/lab3/07_web_server$ ./http_server
Web Server Started on Port 8080...
Request: GET /
Request: GET /favicon.ico
```

10 x 5 계산하기	POST 및 CGI 실행 성공

POST 및 CGI 실행으로 인한 서버 터미널

```
jinho@localhost:~/lab3/07_web_server$ ./http_server
Web Server Started on Port 8080...
Request: GET /
Request: GET /favicon.ico
Request: POST /calc.py
```

8. GUI 관련 함수들을 사용하여 프로그램을 작성하고 실행하여 보고, 익숙해지도록 사용해 본다.

GTK 간단한 버튼 프로그램

```
#include <gtk/gtk.h>
#include <stdio.h>

// 버튼을 클릭했을 때 실행될 콜백 함수
static void print_hello(GtkWidget *widget, gpointer data)
{
    g_print("Hello World Clicked!\n");
}

// 윈도우 닫기 버튼(x)을 눌렀을 때 프로그램을 종료하는 콜백 함수
static void destroy(GtkWidget *widget, gpointer data)
{
    gtk_main_quit();
}

int main(int argc, char *argv[])
{
    GtkWidget *window;
    GtkWidget *button;
```

```

// 1. GTK 환경 초기화
gtk_init(&argc, &argv);

// 2. 윈도우 생성
window = gtk_window_new(GTK_WINDOW_TOPLEVEL);
gtk_window_set_title(GTK_WINDOW(window), "hello_gui");
// 윈도우 크기 및 테두리 설정
gtk_window_set_default_size(GTK_WINDOW(window), 200, 200);
gtk_container_set_border_width(GTK_CONTAINER(window), 10);
// 윈도우 종료 시그널 연결 (창 닫으면 프로그램 종료)
g_signal_connect(window, "destroy", G_CALLBACK(destroy), NULL);
// 3. 버튼 생성
button = gtk_button_new_with_label("Hello World");
// 4. 버튼 시그널 연결
g_signal_connect(button, "clicked", G_CALLBACK(print_hello), NULL);
// 5. 윈도우에 버튼 추가
gtk_container_add(GTK_CONTAINER(window), button);
// 6. 위젯 화면 표시
gtk_widget_show_all(window);
// 7. 메인 루프 실행
gtk_main();

return 0;
}

```

실행	버튼 클릭시 결과 (4번)
	<pre>jinho@localhost:~/lab3/08_gui_basic\$./hello_gui Hello World Clicked! Hello World Clicked! Hello World Clicked! Hello World Clicked!</pre>

GTK+ 팝업 윈도우 프로그램

```

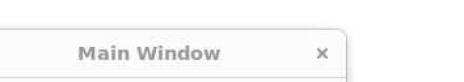
#include <gtk/gtk.h>
#include <stdio.h>

// 메인 윈도우 닫기 콜백
void quit(GtkWidget *window, gpointer data)
{
    gtk_main_quit();
}

int main(int argc, char *argv[])
{
    GtkWidget *window;
    GtkWidget *dialog;
    gint result;

    gtk_init(&argc, &argv);

```

실행	YES 버튼 클릭시 결과
	<pre>jinho@localhost:~/lab3/08_gui_basic\$./gui_dialog 사용자 응답: 예 (YES) -> 축하합니다! 모든 과제를 완료하셨습니다.</pre>
	NO 버튼 클릭시 결과
	<pre>jinho@localhost:~/lab3/08_gui_basic\$./gui_dialog 사용자 응답: 아니오 (NO) -> 조금만 더 힘내세요! 거의 다 왔습니다.</pre>

9. GTK+ 또는 Qt 를 이용하여 간단한 계산기 프로그램을 작성하여 본다

GTK+ 계산기 프로그램

```
#include <gtk/gtk.h>
#include <string.h>
#include <stdlib.h>
#include <stdio.h>

GtkWidget *entry;      // 결과 표시창
int new_input = 1;     // 1이면 다음 입력 시 화면을 새로 씀 (초기화 or 계산 완료 직후)

// [숫자 버튼 핸들러] 화면에 숫자 이어 붙이기
static void on_num_clicked(GtkWidget *widget, gpointer data) {
    const char *text = gtk_button_get_label(GTK_BUTTON(widget));
    const char *current = gtk_entry_get_text(GTK_ENTRY(entry));
    char buffer[256];

    if (new_input) {
        // 새로 입력해야 하는 상태면 기존 내용을 지우고 씀
        strcpy(buffer, text);
        new_input = 0;
    } else {
        // 아니면 뒤에 이어 붙임 (예: "3" -> "3*3")
        // "0"만 있는 상태면 교체
        if (strcmp(current, "0") == 0) strcpy(buffer, text);
        else sprintf(buffer, "%s%s", current, text);
    }
    gtk_entry_set_text(GTK_ENTRY(entry), buffer);
}

// [연산자 버튼 핸들러] 화면에 연산자 이어 붙이기
static void on_op_clicked(GtkWidget *widget, gpointer data) {
    const char *op = gtk_button_get_label(GTK_BUTTON(widget));
    const char *current = gtk_entry_get_text(GTK_ENTRY(entry));
    char buffer[256];

    // 현재 수식 뒤에 연산자 붙이기 (예: "3" -> "3*")
    sprintf(buffer, "%s%s", current, op);
    gtk_entry_set_text(GTK_ENTRY(entry), buffer);

    new_input = 0; // 연산자 뒤에는 숫자가 이어져야 하므로 덮어쓰기 모드 해제
}

// [계산(=) 버튼 핸들러] 화면의 수식을 파싱해서 계산
static void on_calc_clicked(GtkWidget *widget, gpointer data) {
    const char *text = gtk_entry_get_text(GTK_ENTRY(entry));
    char buffer[256];
    strcpy(buffer, text);

    // 연산자 찾기 (+, -, *, /)
    // buffer + 1부터 찾는 이유는 음수(-3)로 시작하는 경우를 대비하기 위함
    char *p = strpbrk(buffer + 1, "+-*/");
```



```

if (p == NULL) return; // 연산자가 없으면 종료

char op = *p; // 연산자 추출
*p = '\0';    // 문자열 분리 (예: "3*3" -> "3"과 "3")

double n1 = atof(buffer); // 앞부분 숫자
double n2 = atof(p + 1);  // 뒷부분 숫자 (연산자 다음 위치)
double result = 0;

switch (op) {
    case '+': result = n1 + n2; break;
    case '-': result = n1 - n2; break;
    case '*': result = n1 * n2; break;
    case '/':
        if(n2 != 0) result = n1 / n2;
        else result = 0;
        break;
}

char res_str[256];
sprintf(res_str, "%.2f", result); // 소수점 2자리 표시
gtk_entry_set_text(GTK_ENTRY(entry), res_str);

new_input = 1; // 계산 완료 후에는 새로운 입력 준비
}

// [초기화(C) 버튼 핸들러]
static void on_clear_clicked(GtkWidget *widget, gpointer data) {
    gtk_entry_set_text(GTK_ENTRY(entry), "0");
    new_input = 1;
}

int main(int argc, char *argv[]) {
    GtkWidget *window;
    GtkWidget *grid;
    GtkWidget *button;

    // 버튼 라벨 배치도 (1-2-3 상단 배치 유지)
    char *buttons[] = {
        "1", "2", "3", "/",
        "4", "5", "6", "*",
        "7", "8", "9", "-",
        "C", "0", "=", "+"
    };

    gtk_init(&argc, &argv);

    // 1. 메인 윈도우 설정
    window = gtk_window_new(GTK_WINDOW_TOPLEVEL);
    gtk_window_set_title(GTK_WINDOW(window), "GTK Calculator");
    gtk_window_set_default_size(GTK_WINDOW(window), 250, 300);
    gtk_container_set_border_width(GTK_CONTAINER(window), 10);

```

```

g_signal_connect(window, "destroy", G_CALLBACK(gtk_main_quit), NULL);

// 2. 그리드(Grid) 레이아웃 생성
grid = gtk_grid_new();
gtk_grid_set_row_spacing(GTK_GRID(grid), 5);
gtk_grid_set_column_spacing(GTK_GRID(grid), 5);
gtk_container_add(GTK_CONTAINER(window), grid);

// 3. 결과 표시창(Entry) 생성
entry = gtk_entry_new();
gtk_entry_set_alignment(GTK_ENTRY(entry), 1); // 오른쪽 정렬
gtk_entry_set_text(GTK_ENTRY(entry), "0");
gtk_editable_set_editable(GTK_EDITABLE(entry), FALSE); // 키보드 입력 방지
gtk_grid_attach(GTK_GRID(grid), entry, 0, 0, 4, 1);

// 4. 버튼 생성 및 배치 루프
for (int i = 0; i < 16; i++) {
    button = gtk_button_new_with_label(buttons[i]);
    int x = i % 4;
    int y = (i / 4) + 1;

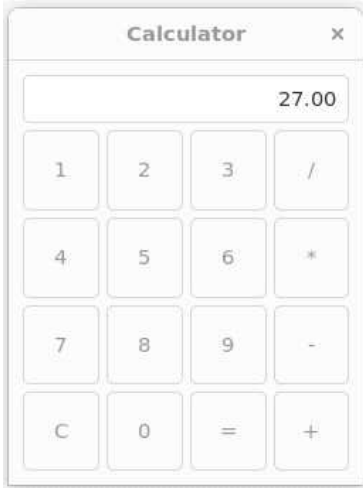
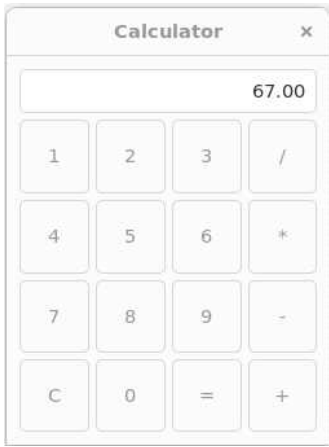
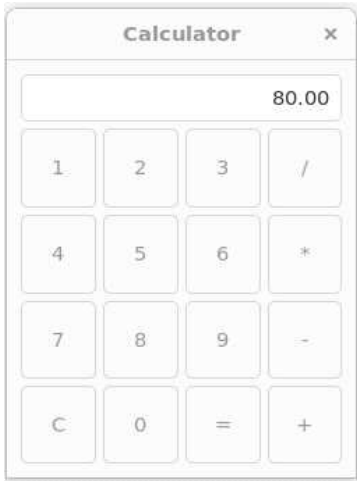
    // 버튼 종류에 따라 다른 핸들러 연결
    if (strcmp(buttons[i], "=") == 0) {
        g_signal_connect(button, "clicked", G_CALLBACK(on_calc_clicked), NULL);
    } else if (strcmp(buttons[i], "C") == 0) {
        g_signal_connect(button, "clicked", G_CALLBACK(on_clear_clicked), NULL);
    } else if (strchr("+-*/", buttons[i][0])) {
        g_signal_connect(button, "clicked", G_CALLBACK(on_op_clicked), NULL);
    } else {
        g_signal_connect(button, "clicked", G_CALLBACK(on_num_clicked), NULL);
    }

    gtk_widget_set_hexpand(button, TRUE);
    gtk_widget_set_vexpand(button, TRUE);
    gtk_grid_attach(GTK_GRID(grid), button, x, y, 1, 1);
}

gtk_widget_show_all(window);
gtk_main();

return 0;
}

```

3 * 3	결과
 A calculator interface with the display showing '3*9'. The keypad includes buttons for digits 1-9, 0, a decimal point, a percentage sign, a square root button, a reciprocal button, a power button, a memory store button, a memory recall button, an equals button, and a plus/minus button.	 A calculator interface with the display showing '27.00'. The keypad is identical to the one in the previous row.
72 - 5	결과
 A calculator interface with the display showing '72-5'. The keypad is identical to the one in the previous row.	 A calculator interface with the display showing '67.00'. The keypad is identical to the one in the previous row.
160 / 2	결과
 A calculator interface with the display showing '160/2'. The keypad is identical to the one in the previous row.	 A calculator interface with the display showing '80.00'. The keypad is identical to the one in the previous row.

검토

이번 실습을 통해 쓰레드, 소켓, GUI 프로그래밍이 각각 따로 존재하는 기술이 아니라, 하나의 프로그램 안에서 유기적으로 결합되는 흐름이라는 것을 체감할 수 있었다. 먼저 POSIX 쓰레드와 뮤텍스, 조건변수, 제한 버퍼 예제를 구현하면서, 이론으로만 배웠던 동기화 기법이 실제로 경쟁 상태를 방지하고 실행 순서를 제어하는 데 얼마나 중요한지 확인하였다. 또한 TCP/UDP 소켓과 select 기반 채팅 서버를 작성하면서, 연결형/비연결형 통신의 차이와 다중 클라이언트를 처리하는 서버 구조를 실습을 통해 익혔다.

또한 간단한 웹 서버를 구현하고 CGI 방식으로 파이썬 프로그램을 연동해 보니, HTTP 요청-응답 구조와 백엔드 프로그램이 어떻게 데이터를 주고받는지 전체 파이프라인을 이해하는 데 큰 도움이 되었고 마지막으로 GTK+를 이용한 버튼, 팝업, 계산기 프로그램을 구현하면서 리눅스 환경에서의 GUI 툴킷 구조와 이벤트 기반 프로그래밍의 기본 흐름을 익힐 수 있었다.

전반적으로 실습 과정은 코드 양이 많고 환경 설정도 까다로웠지만, 교재에서만 보던 개념을 실제 동작하는 프로그램으로 확인했다는 점에서 의미가 컸다.